



American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Stay Finder: A complete Travel Stay Platform

Semester: Spring 25-26

Table of Contents

1.	Project Proposal.....	02
1.1	Background to the Problem	02
1.2	Selection of Process Model	03
2.	Timeline Chart / Gantt Chart	04
3.	PRD	06
3.1	Functional Requirements	07
3.2	Non-Functional Requirements	28
4.	Software Design	37
4.1	System Design	37
4.2	UI Design using Figma	39
5.	Git Workflow	57
6.	Software Testing	61
7.	Conclusion	98

1. PROJECT PROPOSAL

1.1 Background to the Problem

Finding, booking, and managing accommodations can often be complicated for both travelers and property owners. Customers face challenges such as unclear availability, inconsistent pricing, and a lack of trustworthy reviews. On the other hand, property owners struggle to manage multiple listings, track reservations, handle payments securely, and respond to customer feedback.

Existing platforms like Airbnb have shown the demand for short-term rental solutions, but many local businesses and independent hosts still lack a reliable system tailored to their needs. Without a centralized platform, miscommunication, booking errors, and missed business opportunities are common. StayFinder is a web-based software designed to bridge the gap between travelers and property owners by providing a centralized, secure, and intuitive platform for finding, booking, and managing accommodations.

Target Group of Users: The system is designed for multiple groups of users:

- **Customers (Travelers):** Individuals looking for affordable, comfortable, and reliable places to stay, with easy booking and secure payments.
- **Hosts (Property Owners):** People or businesses renting out houses, apartments, or hotels who need tools to manage listings, availability, pricing, and earnings.
- **Super Admin (Platform Owner):** The system administrator who oversees platform operations, ensures quality control, manages users and properties, and earns commission from bookings.

1.2 Selection of Process Model

Process Model Explanation The suitable model for our project is Scrum. Scrum is an Agile software development process model that follows an iterative and incremental approach. The development work is divided into fixed time periods called Sprints, where each Sprint delivers a working product increment. Scrum is adaptive in nature and supports frequent feedback, continuous improvement, and effective collaboration among team members.

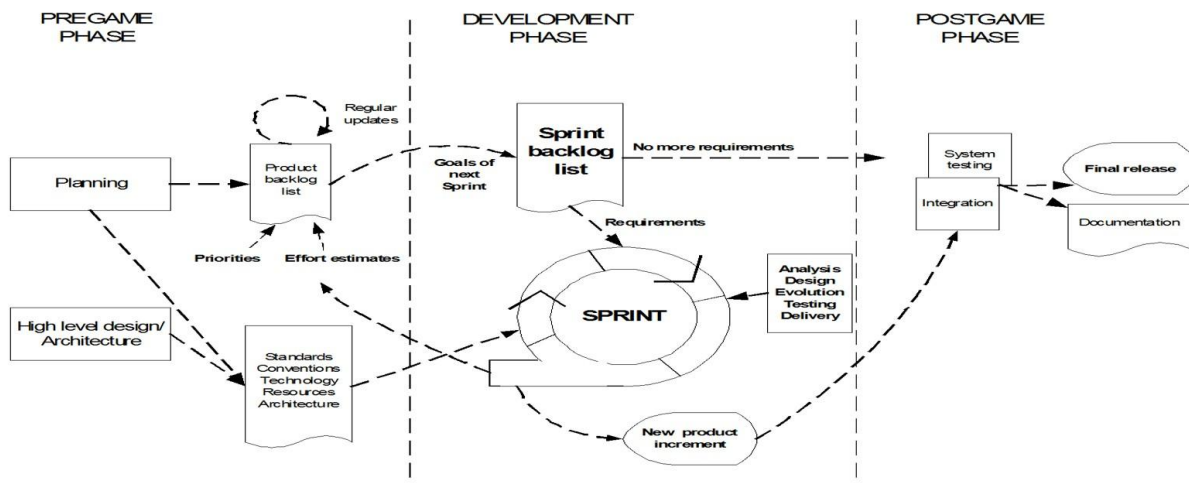
Roles and Responsibilities

- **Scrum Master:** Ensures Scrum rules are followed, facilitates the process, and removes obstacles.

- **Product Owner:** Manages and prioritizes the Product Backlog, makes final decisions on requirements.
- **Scrum Team:** Self-organizing developers who estimate, plan, and deliver increments each sprint.
- **Customer:** Provides input on backlog items and reviews increments to ensure needs are met.
- **Management:** Sets goals, provides resources, and supports standards and decision-making.

Scrum Practices

- **Product Backlog:** A prioritized list of all system requirements, features, and improvements.
- **Sprint:** A fixed-duration development cycle in which selected backlog items are implemented and delivered as a working product increment.
- **Sprint Planning Meeting:** A meeting held at the beginning of each Sprint to define Sprint goals and select Product Backlog items for implementation.
- **Daily Scrum Meeting:** A short daily meeting to track progress, discuss completed work, identify upcoming tasks, and resolve blockers.
- **Sprint Review Meeting:** A meeting conducted at the end of each Sprint to demonstrate the completed product increment and collect stakeholder feedback.



Why We Chose Scrum: We chose the Scrum process model for our project because it offers flexibility and is well suited for building a complex, evolving system. StayFinder has multiple modules (Customer, Host, Admin, Payment). Scrum allows each to be developed and tested in short cycles. Its iterative approach ensures that we can deliver working parts of the system step by step, reducing risks and enabling early problem detection. Scrum's structured practices make it easier to manage changing requirements (e.g., new filters or payment options) without disrupting overall progress.

2. Timeline Chart / Gantt Chart

1. Final Suggestion

Metric	Value
Project Type	Semi-Detached
SLOC	35,000
KLOC	35

2. COCOMO Calculation

2.1 PM Formula

$$PM = 3.0 \times (35)^{1.12}$$

2.2 Development Time

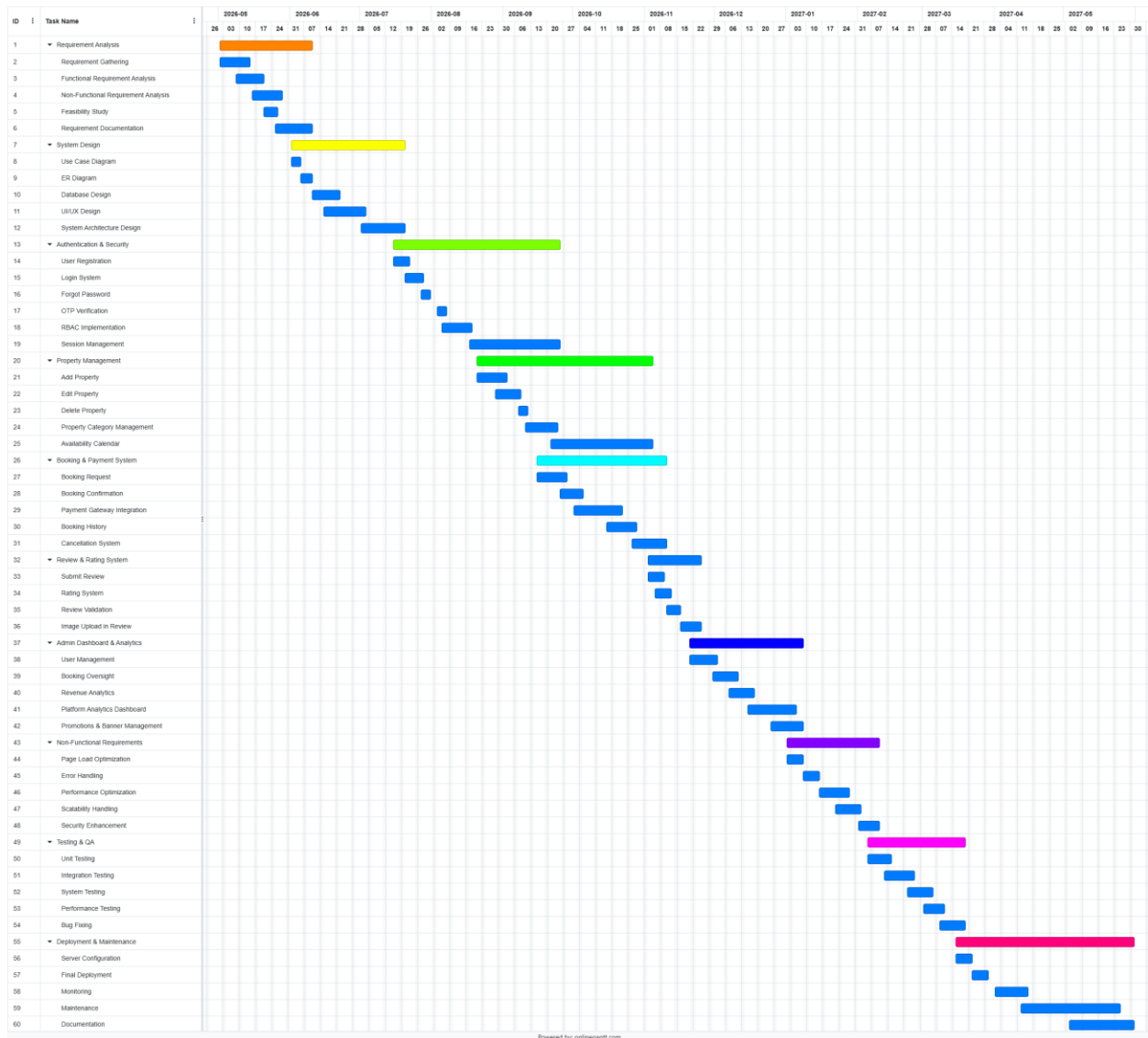
$$TDEV = 2.5 \times (PM)^{0.35}$$

2.3 Team Size

$$ST = PM / TDEV$$

3. Final COCOMO Result

Parameter	Value
PM	160 Person-Month
Development Time	15 Months
Team Size	11 Members



3. REQUIREMENTS DOCUMENT (PRD)

Product Requirement Document (PRD)

StayFinder: A Complete Travel Stay Platform

Client: American International University-Bangladesh (AIUB) – Dept. of Computer Science

Product Type: Web Application

Domain: Travel & Hospitality / Property Rental

Access Layers: Customer, Host, Super Admin

Design Required: Yes – Figma Prototype

Development Phase: Scrum (5 Sprints)

Goals & Objectives: StayFinder aims to bridge the gap between travelers and property owners by providing a centralized, secure, and intuitive platform for finding, booking, and managing accommodations. User roles: Customer (Traveler) – Search, book, and pay for properties; leave reviews. Host (Property Owner) – List properties,

manage bookings, track earnings. Super Admin – Oversee platform, approve listings, manage users, track commission.

Functional Requirements

Phase	Item No	Specification [Role]	Screen Definition	User Story	Acceptance Criteria
Feature 1: Authentication & Profile Management					
Phase 1	1	The system should allow customers to register for a new account [Customer]	Header: Platform logo + navigation Content: - Registration form for customers - Fields: Full Name, Email, Username, Password, Confirm Password, Phone Number, Address - OTP/Email verification option CTA: “Register” button Footer: Terms & Conditions, Privacy Policy	As a user, I want to register as a customer so that I can create an account and access the platform services.	- Full name, email, username, phone, DOB, address, postcode, city, and division should be required. - Email (input) field should only accept a valid and unique email address. Otherwise, the system should show an 'invalid or already used email' message. - Password (input) should be minimum 8 characters long. - Confirm password (input) should match password. Otherwise, the system should show a 'password do not match' message. - OTP should be sent to phone number for verification.
					- A confirmation email should be sent after successful registration. - On clicking register button, customer information should be stored in database and user should be redirected to customer dashboard.

Phase 1	2	The system should allow hosts to register for a new account [Host]	Header: Platform logo + navigation Content: - Registration form for hosts - Fields: Full Name/Business Name, Email, Username, Password, Confirm Password, Phone Number, Business Address - Option for uploading property/business details (optional at this stage) - OTP/Email verification step CTA: "Register as Host" button Footer: Terms & Conditions, Privacy Policy links	As a property owner (host), I want to register on the platform so that I can list and manage my properties and start receiving bookings.	- The system should require all mandatory fields before allowing submission - The system should verify the user's email/phone using OTP - The system should enforce password rules (minimum length, special characters, etc.) - The system should display a success message after successful registration - The system should redirect the user to the host dashboard after registration - The system should show appropriate error messages for invalid or duplicate inputs
Phase 1	3	The system should allow users to sign in to their accounts [Customer/Host]	Header: Platform logo + navigation Content: Sign in form for customers and hosts	As a user, I want to sign in securely so that I can access my account and dashboard.	- Email and password should match database records. - On successful signin, user should be redirected to the appropriate

			Fields: - Email - Password - Remember Me Options: - Google Sign In - Forgot Password CTA: Sign In Footer: Terms & Conditions, Privacy Policy links		dashboard (customer / host). - If credentials do not match, the system should show 'Invalid email or password' message. - Google sign-in option should be available. - Remember me option should store session cookie for 30 days.
Phase 1	4	The system should allow admin users to sign in to their accounts [Admin]	Header: Platform logo + admin portal title Content: Admin sign in form Fields: - Admin Email - Password Options: - Forgot Password CTA: Sign In Footer: Security notice / Privacy Policy	As a user, I want to sign in as an admin so that I can access the admin dashboard and manage platform operations.	- Email (input) field should end with @stayfinder.com. Otherwise, the system should show 'Invalid admin email' message. - Password (input) should match database record. - On successful signin, user should be redirected to admin dashboard. - Failed login attempts after 5 tries should lock the account for 15 minutes.
Phase 1	5	The system should allow users to reset their password if	Header: Platform logo + navigation	As a user, I want to reset my password	- Email (input) field should exist in database. Otherwise, the system

		they forget it [Customer/Host/Admin]	Content: Password recovery form Fields: - Email - OTP - New Password - Confirm Password CTA: - Send OTP - Reset Password Footer: Back to Sign In link	so that I can regain access to my account if I forget my login credentials.	should show 'No account found' message. - OTP should be sent to registered email address and it should remain valid for 10 minutes. - New password (input) should be minimum 8 characters long and should contain at least one special character. - Confirm password (input) should match new password. Otherwise, the system should show 'password do not match' message. - On successful password reset, system should show 'Password reset successful' message and user should be redirected to sign in page.
--	--	--	---	--	--

Phase 1	6	The system should allow users to view and edit their profile information [Customer/Host]	Header: Platform logo + user navigation Content: Profile page with editable form Fields: - Full Name - Email - Phone - Date of Birth - Address - Profile Photo - Change Password Section	As a user, I want to view and edit my profile information so that I can keep my account details updated.	- All existing profile fields should be pre-filled in edit form. - Email format should be validated and phone number should be 11 digits. - On clicking save, success notification should be shown and profile page should be refreshed. - Change photo should accept PNG/JPG only and maximum file size should be 2MB. - For password change, old password should be verified and new password should be
---------	---	---	--	---	---

			CTA: - Save Changes - Update Photo - Change Password		minimum 8 characters long.
--	--	--	---	--	-----------------------------------

Feature 2: Property Management

Phase 2	7	The system should allow hosts to add new property listings [Host]	Header: Host dashboard + navigation Content: Property listing creation form Fields: - Property Title - Description - Property Type - Category - Price Per Night - Location - Photos - Amenities - Availability CTA: Submit Listing Footer: Listing guidelines / help link	As a user, I want to add a property listing so that customers can view and book my property.	- All required fields (title, location, price, type, category) should be filled. - Minimum 1 photo should be required and each photo size should be maximum 5MB. - Price should be a positive number. - Property should be saved with 'Pending Approval' status. - Host should receive notification when admin approves or rejects the property. - Listing should be visible on platform only after admin approval.
---------	---	--	--	---	--

Phase 2	8	The system should allow hosts to edit or remove their property listings [Host]	Header: Host dashboard + navigation Content: Property management page Sections: - Existing Listings Table - Edit Listing Form - Remove Listing Action Fields: - Title - Description - Type - Category - Price - Location - Photos - Amenities CTA: - Save Changes - Remove Listing	As a user, I want to edit or remove my property listing so that I can keep my property information accurate and updated.	- Edit form should be pre-filled with existing data. - Any edited listing should return to 'Pending Approval' status. - Remove action should require confirmation dialog. - Removed listing should disappear from customer search immediately. - Host should be able to view all listings with status (active / pending / rejected).
---------	---	---	--	---	--

Phase 2	9	The system should allow administrators to approve or reject property listings [Admin]	Header: Admin dashboard + navigation Content: Pending listings review page Sections:	As a user, I want to review submitted property listings so that only valid and approved listings appear on the platform.	- Pending listings table should show all submissions awaiting review. - On approving a listing, listing status should change to 'Active' and host should be notified by email.
---------	---	--	---	--	---

			- Pending Listings Table - Listing Details Preview - Approval / Rejection Controls Fields: - Listing Information - Host Information - Rejection Reason CTA: - Approve - Reject - Filter by Status / Date / Category		- On rejecting a listing, admin should enter rejection reason and host should be notified with the reason. - Approved listings should immediately appear in customer search. - Admin should be able to filter listings by status, date, and category.
--	--	--	---	--	---

Phase 2	10	The system should allow administrators to manage property categories and types [Admin]	Header: Admin dashboard + navigation Content: Category and type management page Sections: - Category List - Property Type List - Add / Edit / Delete Controls Fields: - Category Name - Type Name	As a user, I want to manage property categories and types so that listings can be organized properly across the platform.	- Admin should be able to add new property categories and types. - Admin should be able to edit existing property categories and types. - Admin should be able to delete unused categories and types. - Updated categories and types should be reflected in listing forms and search filters. - Duplicate category or type names should not be allowed.
---------	----	---	---	--	---

			- Status CTA: - Add Category - Add Type - Edit - Delete		
Feature 3: Property Search & Discovery					

Phase 2	11	Search & Filter Properties [Customer]	Header: Platform logo + search bar + navigation Content: - Search inputs: Location, Check-in & Check-out dates, Guests - Filters: Price range, Property type (Hotel/Villa/Home), Category (Luxury/Premium/Regular), Amenities, Ratings - Search results section displaying property cards (image, price, rating, location) - Sorting options (price low-high, rating, popularity) CTA: “Search” / “Apply Filters” button	As a customer, I want to search and filter properties so that I can find suitable accommodation based on my preferences.	- The system should allow users to search properties by location, date, and guest count - The system should provide filters for price, category, type, amenities, and ratings - The system should display matching properties based on selected filters - The system should update results dynamically after applying filters - The system should allow sorting of results (price, rating, etc.) - The system should show a message if no properties match the criteria
---------	----	--	--	---	--

Phase 2	12	View Property Details [Customer]	Header: Platform logo + navigation Content: - Property title, images/gallery - Price per night - Property details (type, category, location, description) - Amenities list (WiFi, AC, etc.) - Availability calendar - Host information - Ratings and reviews section CTA: “Add to Cart” / “Book Now” button	As a customer, I want to view detailed information about a property so that I can decide whether to book it.	- The system should display complete property details including price, location, and description - The system should show property images and amenities - The system should display availability calendar for selected dates - The system should show ratings and customer reviews - The system should provide host information - The system should allow users to proceed to booking or add to cart
---------	----	---	---	---	---

Phase 3	13	Offers & Discounts Page [Customer]	Header: Platform logo + navigation Content: - Section header: “Special Offers & Discounts” - List of offers (cards with image, title, discount %, validity) - Categories: Seasonal offers, promo codes, special packages - Option to apply promo code	As a customer, I want to view available offers and discounts so that I can save money on my bookings.	- The system should display all available offers and discounts - The system should show details of each offer (discount amount, validity, conditions) - The system should allow users to apply promo codes - The system should validate promo codes before applying discounts - The system should update the booking price after applying a valid discount
---------	----	---	---	--	--

			- Highlighted featured deals CTA: “Apply Offer” / “View Deal” button		- The system should show an error message for invalid or expired offers
--	--	--	---	--	---

Feature 4: Cart & Booking Management

Phase 3	14	Cart Management [Customer]	Header: Platform logo + navigation Content: - Section header: “Special Offers & Discounts” - List of offers (cards with image, title, discount %, validity) - Categories: Seasonal offers, promo codes, special packages - Option to apply promo code - Highlighted featured deals CTA: “Apply Offer” / “View Deal” button	As a customer, I want to view available offers and discounts so that I can save money on my bookings.	- The system should display all available offers and discounts - The system should show details of each offer (discount amount, validity, conditions) - The system should allow users to apply promo codes - The system should validate promo codes before applying discounts - The system should update the booking price after applying a valid discount - The system should show an error message for invalid or expired offers
Phase 3	15	Create Booking with Partial Payment [Customer]	Header: Platform logo + navigation Content: - Booking summary (property details, dates, guests, total price)	As a customer, I want to create a booking with partial payment so that I can reserve a property without paying	- The system should allow users to book a property by paying a partial amount - The system should calculate and display the partial payment and remaining balance

			- Payment section showing partial payment amount (e.g., 30%) - Option to choose payment method - Display of remaining balance - Booking status indicator CTA: “Confirm Booking” / “Pay Partial Amount” button	the full amount immediately.	- The system should confirm booking after successful partial payment - The system should update booking status as “Partial Paid” - The system should store booking details in booking history - The system should allow users to complete the remaining payment before check-in
Phase 3	16	Complete Remaining Payment [Customer]	Header: Platform logo + navigation Content: - Booking details (property, dates, total amount) - Payment summary (paid amount, remaining balance) - Payment method selection - Payment status indicator CTA: “Pay Remaining Amount” button	As a customer, I want to complete the remaining payment so that I can confirm my booking before check-in.	- The system should display the remaining balance for a partially paid booking - The system should allow users to pay the remaining amount - The system should update the booking status to “Fully Paid” after successful payment - The system should generate a payment confirmation notification - The system should update booking history with full payment status - The system should prevent duplicate or overpayment

Phase 3	17	View Booking History [Customer]	Header: Platform logo + navigation Content:	As a customer, I want to view my booking history so that I can track my past and	- The system should display a list of all bookings for the user - The system should categorize bookings as past and upcoming
---------	----	--	--	--	---

			- Booking history table/list - Sections: Past bookings and Upcoming bookings - Each booking shows: Property name, dates, price, payment status, booking status - Filter/sort options (date, status) CTA: "View Details" button for each booking	upcoming stays.	- The system should show booking details (property, date, price, status) - The system should display payment status (partial/fully paid) - The system should allow users to view detailed information of each booking - The system should allow filtering or sorting of booking history
--	--	--	---	-----------------	--

Feature 5: Reviews & Ratings

Phase 3	18	Customer should submit Property Review [Customer]	- Header: Platform logo + navigation - Content: - Section Header: "Submit Review" - Review Form: - Property Name (autofilled) - Rating (1–5 stars) - Comment box - Upload images (optional) - CTA: - Submit Review button - Confirmation message after submission	As a customer who completed a stay, I want to rate and review the property so that I can help other travelers make informed decisions.	- Customer should be able to submit a review for a property - Rating should be selectable from 1 to 5 stars - Review text should be required before submission - Images should be optional to upload - Confirmation message should be shown after successful submission

Phase 3	19	The system should allow host to reply the customer reviews [Host]	Header: Platform logo + host navigation Content: - Section Header: “Review Responses” - Review List: - Customer Name - Rating - Review text - Date - Reply Box under each review - Actions: - Submit Reply - Edit / Delete Reply	As a host, I want to respond to customer reviews so that I can address feedback and engage with guests.	- Host should be able to view all customer reviews - Host should be able to reply to a review - Reply should be editable or deletable by host - Reply should be displayed under the original review - Changes should be updated instantly
Phase 3	20	The system should allow admin to review moderation [Admin]	Header: Platform logo + admin navigation Content: - Section Header: “Review Moderation” - Review Table: - Review ID - User Name - Property Name - Rating - Comment	As the admin, I want to moderate reviews and handle complaints so that the platform maintains trust and quality.	- Admin should be able to view all submitted reviews - Admin should be able to approve or reject reviews - Review status should be updated (Pending/Approved/Rejected) Admin should be able to delete inappropriate reviews - Only approved reviews should be visible to users

			- Status (Pending/Approved/Rejected) - Filters: - Status - Date - Actions: - Approve / Reject Review - Delete Review		
Feature 6: Host Dashboard & Earnings					

Phase 2	21	The system should allow host to manage booking Request [Host]	Header: Platform logo + host navigation Content: - Section Header: “Booking Requests” - Booking List: - Request ID - Guest Name - Property Name - Dates - Status - Actions: - Accept / Reject Request - View Details - Filters: - Date - Status	As a host, I want to review and respond to booking requests so that I can manage my property reservations efficiently.	- Host should be able to view all booking requests - Host should be able to accept or reject requests - Booking status should be updated after action - Host should be able to view request details - Notifications should be sent to customer after decision
---------	----	--	---	---	---

Phase 2	22	The host should have a dashboard for Earnings & Analytics [Host]	Header: Platform logo + host navigation Content: - Section Header: “Earnings & Analytics” - Summary Cards: - Total Earnings - Monthly Earnings - Total Bookings - Charts: - Earnings over time - Booking trends - Filters: - Date range	As a host, I want to view my earnings and booking analytics so that I can track my business performance.	- Host should be able to view total and monthly earnings - Earnings data should be displayed in charts - Booking trends should be visible - Data should be filterable by date range - Information should be accurate and updated regularly
Phase 2	23	The Host should have Availability Calendar for Management [Host]	Header: Platform logo + host navigation Content: - Section Header: “Availability Calendar” - Calendar View: - Monthly/weekly view - Available / Booked dates - Actions: - Mark dates as available/unavailable - Edit availability - Legend for status colors	As a host, I want to manage my property's availability calendar so that customers only see accurate available dates.	- Host should be able to view availability in calendar format - Host should be able to mark dates as available or unavailable - Changes should be saved and reflected immediately - Booked dates should be shown clearly - Calendar should support monthly and weekly views

Feature 7: Admin Dashboard & Platform Management

Phase 4	24	User Management [Admin]	Header: Platform logo + admin navigation Content: - Section Header: "User Management" - User Table: - User ID - Name - Email - Role (Customer/Host/Admin) - Status (Active/Blocked) - Filters: - Role - Status - Actions: - View Details - Edit User - Activate / Deactivate - Delete User	As the admin, I want to view and manage all customer and host accounts so that I can maintain platform integrity.	- Admin should be able to view all users - Admin should be able to edit user information - Admin should be able to activate or deactivate users - Admin should be able to delete users - User roles should be clearly displayed
---------	----	--------------------------------	--	--	---

Phase 4	25	The system should allow admin to booking & payment oversight [Admin]	Header: Platform logo + admin navigation Content: - Section Header: "Booking & Payment Oversight" Booking Management:	As an admin, I want to monitor and control bookings and payments so that I can ensure smooth operations and accurate transactions.	- Admin should be able to view all bookings and payment records - Booking status should be updated (approve/reject/cancel) - Payment status should be tracked (paid/pending/failed) - Admin should be able to verify and update payments
---------	----	---	--	---	---

			- Booking Table: - Booking ID - Customer Name - Service Name - Date & Time - Status (Pending/Confirmed/Canceled) - Filters: - Date - Status - Actions: - View Details - Approve / Reject Booking - Cancel Booking Payment Management: - Payment Table: - Transaction ID - Booking ID - Customer Name - Payment Method - Amount - Status (Paid/Pending/Failed) - - Actions: - Verify Payment - Mark as Paid - Refund		- Summary data should be updated in real-time
--	--	--	--	--	---

			Summary Panel: - Total bookings - Total payments - Pending payments		
--	--	--	--	--	--

Phase 4	26	The Admin should have a platform Analytics Dashboard [Admin]	Header: Platform logo + admin navigation Content: - Section Header: "Platform Analytics Dashboard" Overview Cards: - Total Users - Total Bookings - Total Revenue - Active Users Booking Analytics: - Chart (daily/weekly/monthly bookings) - Filters: - Date range - Booking status	As an admin, I want to view analytics and insights so that I can make informed decisions about the platform.	- Dashboard should display real-time analytics data - Charts should update dynamically based on filters - Data should be accurate and consistent across all sections - Admin should be able to filter data by date and category - Key metrics should be visible at a glance
---------	----	---	--	---	---

			Revenue Analytics: - Revenue trend chart - Revenue by category/service User Analytics: - New users - Active vs inactive users - Growth chart Payment Analytics: - Total transactions - Successful vs failed payments Insights Section: - Most popular service - Peak booking time		
--	--	--	--	--	--

Phase 4	27	Promotions & Banner Management [Admin]	Header: Platform logo + admin navigation Content: - Section Header: “Manage Promotions & Banners” - Banner Table:	As an admin, I want to manage promotions and banners so that I can attract users and increase engagement.	- Admin should be able to add, edit, and delete banners - Image upload should support JPG/PNG (≤2MB) - Promotions should activate automatically based on schedule
---------	----	---	---	--	---

			- Banner ID, Title, Image, Start Date, End Date, Status - Actions: - Add / Edit / Delete Banner - Activate / Deactivate - Promotion - Setup: - Discount (%) - Service selection - Validity period - Live Preview panel for banners		- Changes should reflect on user side within 5 seconds
--	--	--	--	--	--

Feature 8: Notifications & Communication

Phase 3	28	Booking & Payment Notifications [Customer/Host/Admin]	Header: Platform logo + admin navigation Content: - Section Header: "Notification Management" - Settings: - Booking notification toggle - Payment success/failure notification toggle - Notification Log: - Notification ID, User, Type, Message, Status, Time - Actions: - Resend notification - Edit message templates	As an admin, I want to control and monitor notifications so that users get timely updates about bookings and payments.	- Notifications should be sent instantly after booking/payment events - Failed notifications should be logged - Admin should be able to resend notifications - Message templates should be editable
---------	----	--	---	---	--

Phase 3	29	Complaint & Escalation System [Admin]	Header: Platform logo + admin navigation Content: - Section Header: "Complaint Management" - Complaint List: - Complaint ID, User Name, Issue Type, Status - Filters: - Status, Date - Actions: - View details - Assign to staff - Escalate complaint - Mark as resolved - Escalation Panel: - Priority level - Notes	As an admin, I want to manage complaints and escalate important issues so that problems are resolved efficiently.	- Admin should be able to view and update complaint status - Escalation option should be available for unresolved issues - Notifications should be sent when status changes - Complaint history should be stored
---------	----	--	---	--	---

Phase 4	30	10% Commission Auto-Calculation [Admin]	Header: Platform logo + admin navigation Content: Section Header: “Commission Management” Transaction Table: Booking ID, Total Amount, Commission (10%), Net Amount Auto-calculation display Filters:	As an admin, I want the system to automatically calculate commission so that financial records remain accurate.	Commission should be calculated automatically for each transaction Calculation should be accurate (10% of total amount) Values should update instantly after payment Reports should include commission details
---------	----	--	--	--	--

			Date, Payment status Summary: Total revenue Total commission		
Phase / Category	Item No	Specification [Role/Category]	Screen Definition	User Story	Acceptance Criteria

Performance	1	Page Load Time [Performance]	System-wide performance requirement Applies to: - Search results page - Home page - Property details page - Dashboard pages - Other user-facing pages	As a user, I want pages to load quickly so that I can use the platform smoothly without delay.	- Search results page should load within 3 seconds under normal operating conditions. - Other user-facing pages should load within 2 seconds under normal operating conditions. - If page load exceeds defined threshold, the system should log the performance issue. - Performance should remain within acceptable range during standard user activity.
Performance	2	Concurrent User Capacity [Performance]	System-wide scalability and performance requirement Applies to: - Customer portal - Host portal - Admin portal - Search and booking services	As a user, I want the platform to remain stable during heavy traffic so that I can use it without interruption.	- The system should support at least 1,000 concurrent users. - The system should remain responsive without major slowdown during concurrent access. - High concurrent usage should not cause system crash or service unavailability.
					- Performance metrics should be monitored during peak usage.

Performance	3	Booking Confirmation Response [Performance]	Booking and payment processing module Applies to: - Checkout page - Payment confirmation page - Booking summary page	As a user, I want booking confirmation to appear quickly after payment so that I know my booking was successful.	- Booking confirmation should be displayed within 5 seconds after successful payment completion. - Confirmation message should include booking status and summary. - If delay occurs, the system should show a processing status message. - Booking record should be stored successfully before confirmation is displayed.
Scalability	4	Horizontal Scalability [Scalability]	Infrastructure and deployment architecture requirement Applies to: - Application servers - Web services - Search and booking modules	As a user, I want the platform to handle traffic growth efficiently so that service quality remains stable over time.	- The system should support horizontal scaling by adding additional service instances. - Increased traffic should be distributed across multiple running instances. - Scaling process should not require major changes to application logic. - System availability should be maintained during scaling operations.
Scalability	5	Database Scalability [Scalability]	Database architecture requirement Applies to: - User records	As a user, I want the system to handle growing data efficiently so that performance remains reliable as the	- The database should support growth in users, bookings, and property listings. - Query performance should remain stable as record volume increases.

			- Property records - Booking records - Payment records - Review records	platform expands.	- Data storage design should support future expansion. - Large-scale data growth should not significantly affect transaction processing.
Security	6	Data Encryption [Security]	Header: Platform logo + secure navigation Content: - Secure login and authentication interface (OTP/email verification) - Encrypted data transmission layer (HTTPS/SSL) - Secure storage system for user data (passwords, payments) - Payment gateway integration with encryption - Background security processes (data hashing, tokenization)	As a user, I want my personal and payment data to be encrypted so that my information remains secure and protected from unauthorized access.	- The system should encrypt all sensitive user data during transmission using secure protocols (HTTPS/SSL) - The system should encrypt sensitive data (passwords, payment details) in storage - The system should use hashing algorithms for password protection - The system should ensure secure communication between client and server - The system should protect data from unauthorized access or breaches - The system should follow standard security practices for data encryption

Security	7	Role-Based Access Control (RBAC) [Security]	Header: Platform logo + role-based navigation Content: - Login system with role identification (Customer / Host / Admin)	As a system user, I want access to features based on my role so that I can only view and perform actions relevant to my permissions.	- The system should assign roles (Customer, Host, Admin) to users upon registration/login - The system should restrict access to features based on user roles - The system should prevent unauthorized
----------	---	--	--	--	--

			- Dynamic dashboard based on user role - Restricted access views (Customer, Host, Admin panels) - Permission management layer controlling feature access - Unauthorized access handling (error/unpql messages or redirects)		- users from accessing restricted pages - The system should display different dashboards based on user roles - The system should validate user permissions before performing any action - The system should redirect or show an error message for unauthorized access attempts - The system should ensure secure separation of functionalities between roles
--	--	--	--	--	--

Security	8	OTP & Email Verification [Security]	Header: Platform logo + authentication interface Content: - Registration/Login verification screen - Input fields for OTP and email verification code - “Send OTP” / “Resend OTP” options - Email verification link handling (click-to-verify) - Timer/countdown for OTP validity - Success and error notification messages	As a user, I want to verify my account using OTP or email so that my account is secure and authenticated.	- The system should send an OTP to the user’s registered email or phone number during registration/login - The system should allow users to enter and verify OTP within a limited time - The system should provide a resend OTP option after a specific interval - The system should verify user email through a secure verification link - The system should block access if OTP/email verification is not completed - The system should display appropriate success or error messages during verification
					- The system should expire OTP after a defined time period to ensure security

Security	9	Payment Security Compliance [Security]	Header: Platform logo + secure payment navigation Content: - Secure payment page with payment method selection (card, mobile banking, etc.) - Integration with trusted payment gateway - Encrypted payment form (card details, transaction info) - Payment status screen (success, failed, pending) - Transaction logs and confirmation interface - Security indicators (SSL badge, secure payment notice)	As a customer, I want my payment transactions to follow security standards so that my financial data remains safe and trustworthy.	- The system should comply with standard payment security protocols (e.g., PCI DSS) - The system should encrypt all paymentrelated data during transmission - The system should not store sensitive payment details (e.g., full card number, CVV) - The system should use a secure and trusted payment gateway for processing transactions - The system should validate all payment transactions before confirmation - The system should provide secure payment status updates (success, failed, pending) - The system should protect against fraudulent or unauthorized transactions
Security	10	Account Lockout Policy [Security]	Header: Platform logo + secure login interface Content: - Login page with username/email and password fields - Failed login attempt tracking mechanism	As a user, I want my account to be temporarily locked after multiple failed login attempts so that my account is protected from unauthorized access.	- The system should track consecutive failed login attempts - The system should lock the user account after a predefined number of failed attempts - The system should notify the user when the account is locked

			- Account lock notification message screen - “Forgot Password” / account recovery option - Timer or countdown showing lock duration - Security alert/notification system (email/SMS for suspicious activity)		- The system should allow account recovery through password reset or verification - The system should automatically unlock the account after a specified time or admin intervention - The system should reset the failed attempt counter after a successful login - The system should send a security alert to the user for suspicious login activity
Availability	11	System Uptime [Availability]	Section Header: “System Availability Status” Components: - Live uptime percentage indicator (e.g., 99.9%) - Server status (Online/Offline) - Downtime history log (last 24h/7d) - Maintenance notification banner	99% uptime SLA (≤ 7.2 hours downtime per month). Monitored via uptime tool.	- The system should maintain uptime of at least 99.9% - The system should display real-time server status - The system should notify users before scheduled maintenance - The system should log and display downtime history accurately

Availability	12	Automatic Failover & Recovery [Availability]	Section Header: “System Reliability Status” Components:	Recovery Time Objective (RTO) ≤10 minutes. Recovery Point	- The system should automatically trigger failover within 5 seconds of failure
--------------	----	---	--	--	--

			- Server status indicator (Active/Backup) - Failover alert popup - Recovery progress bar	Objective (RPO) ≤1 hour.	- The system should minimize downtime to less than 10 seconds - The system should restore user sessions after recovery
Availability	13	Data Backup [Availability]	Section Header: “Backup & Restore” Components: - Backup status (Last backup time) - Manual backup button - Restore option with confirmation modal	Automated daily backups; backup stored in geographically separate location; 30day retention.	- The system should perform automatic daily backups - The system should ensure 100% successful backup completion - The system should restore data within 15 seconds when requested

Usability	14	Responsive UI Design [Usability]	Section Header: “Adaptive Layout” Components: - Mobile, Tablet, Desktop views - Flexible grid layout - Collapsible menu (mobile)	Layout adapts to mobile (320px), tablet (768px), desktop (1280px+). No horizontal scroll on mobile.	- The system should adapt UI layouts for mobile, tablet, and desktop screens - The system should prevent horizontal scrolling on mobile devices - The system should load all views within 3 seconds
-----------	----	---	--	--	---

Usability	15	Cross-Browser Compatibility [Usability]	Section Header: “Browser Support” Components: - Compatibility info section - UI consistency across browsers	Fully functional on Chrome 110+, Firefox 110+, Edge 110+, Safari 16+. Zero critical bugs across browsers.	- The system should function correctly on Chrome, Firefox, and Edge browsers - The system should maintain consistent UI across supported browsers - The system should ensure all core features work without errors
-----------	----	--	--	--	--

Usability	16	Error Messages & Validation [Usability]	Section Header: “Form Validation” Components: - Inline error messages - Field highlight (red border) - Tooltip guidance	100% of required fields validated client-side + server-side. Error messages in plain English, shown inline.	- The system should display validation errors instantly (within 1 second) - The system should provide clear and userfriendly error messages - The system should prevent form submission with invalid inputs
Maintainability	17	Modular Code Architecture [Maintainability]	Section Header: “System Architecture Overview” Components: - Module breakdown diagram - API interaction layer - Component-based UI structure	Each module (Auth, Booking, Property, Payment, Admin) independently deployable. Documented API contracts.	- The system should support independent module deployment - The system should allow code reuse across multiple components - The system should ensure that changes in one module do not affect others

--	--	--	--	--	--

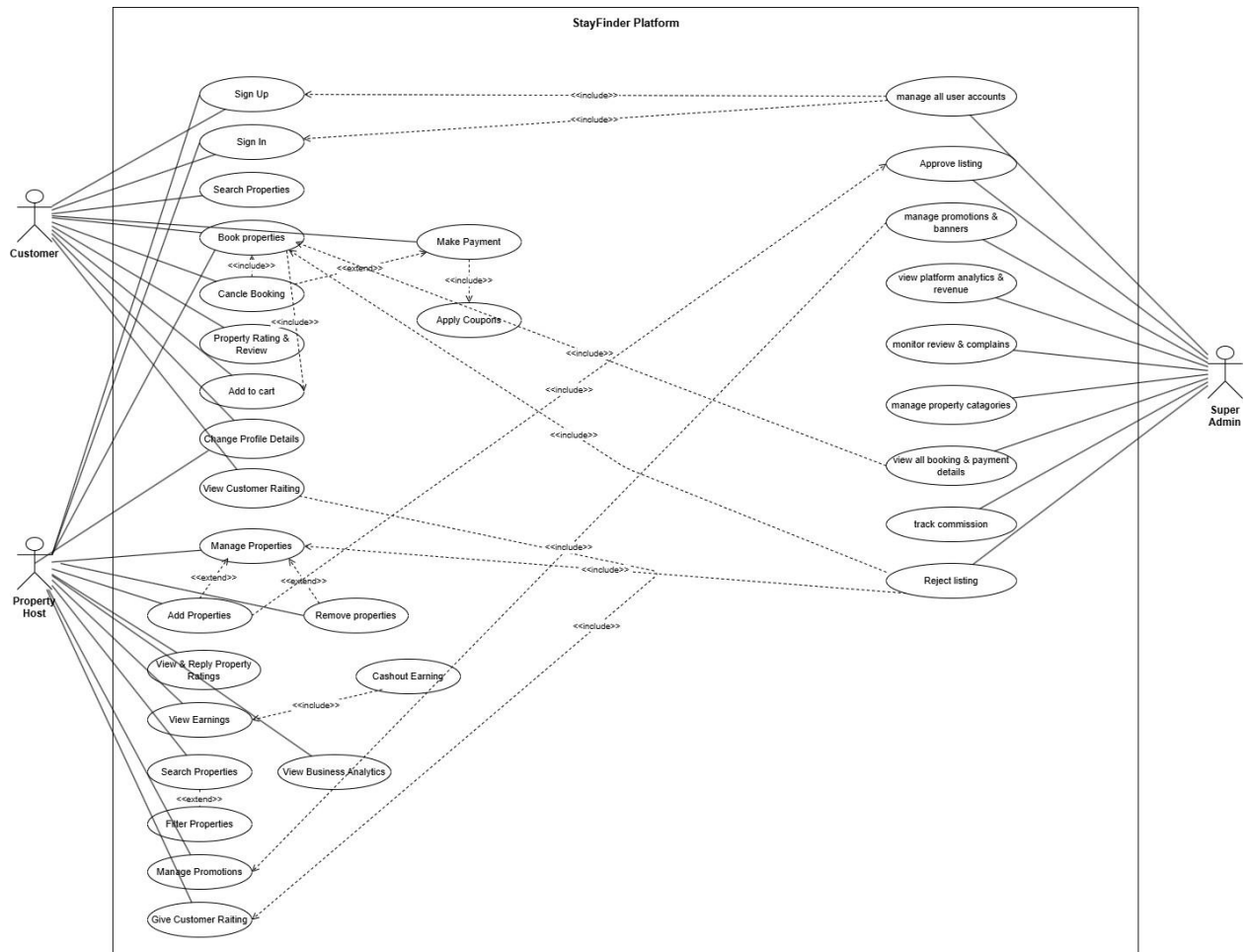
Maintainability	18	Admin Self-Service Management [Maintainability]	Section Header: “Admin Control Panel” Components: - User management dashboard - Listing approval system - Settings configuration panel	100% of content management tasks completable via admin UI without code changes.	- The system should allow admins to manage users and listings independently - The system should apply admin actions instantly - The system should enforce role-based access control
Portability	19	Cloud Deployment Compatibility [Portability]	Section Header: “Cloud Environment Setup” Components: - Deployment status dashboard - Server region selector - CI/CD pipeline indicator	Application deployable on AWS / GCP / Azure. Docker container support. No OS-specific dependencies.	- The system should support deployment on AWS, GCP, and Azure - The system should complete deployment within 5 minutes - The system should enable automatic scaling based on load
Data Integrity	20	Transactional Data Consistency [Data Integrity]	Section Header: “Transaction Monitoring” Components: - Booking transaction log - Payment status tracker - Conflict resolution alerts	Zero duplicate bookings for same property/dates. No orphaned payments. ACID transactions for booking+payment operations.	- The system should prevent duplicate transactions - The system should synchronize transactional data in real-time (within 2 seconds) - The system should rollback incomplete transactions automatically

--	--	--	--	--	--

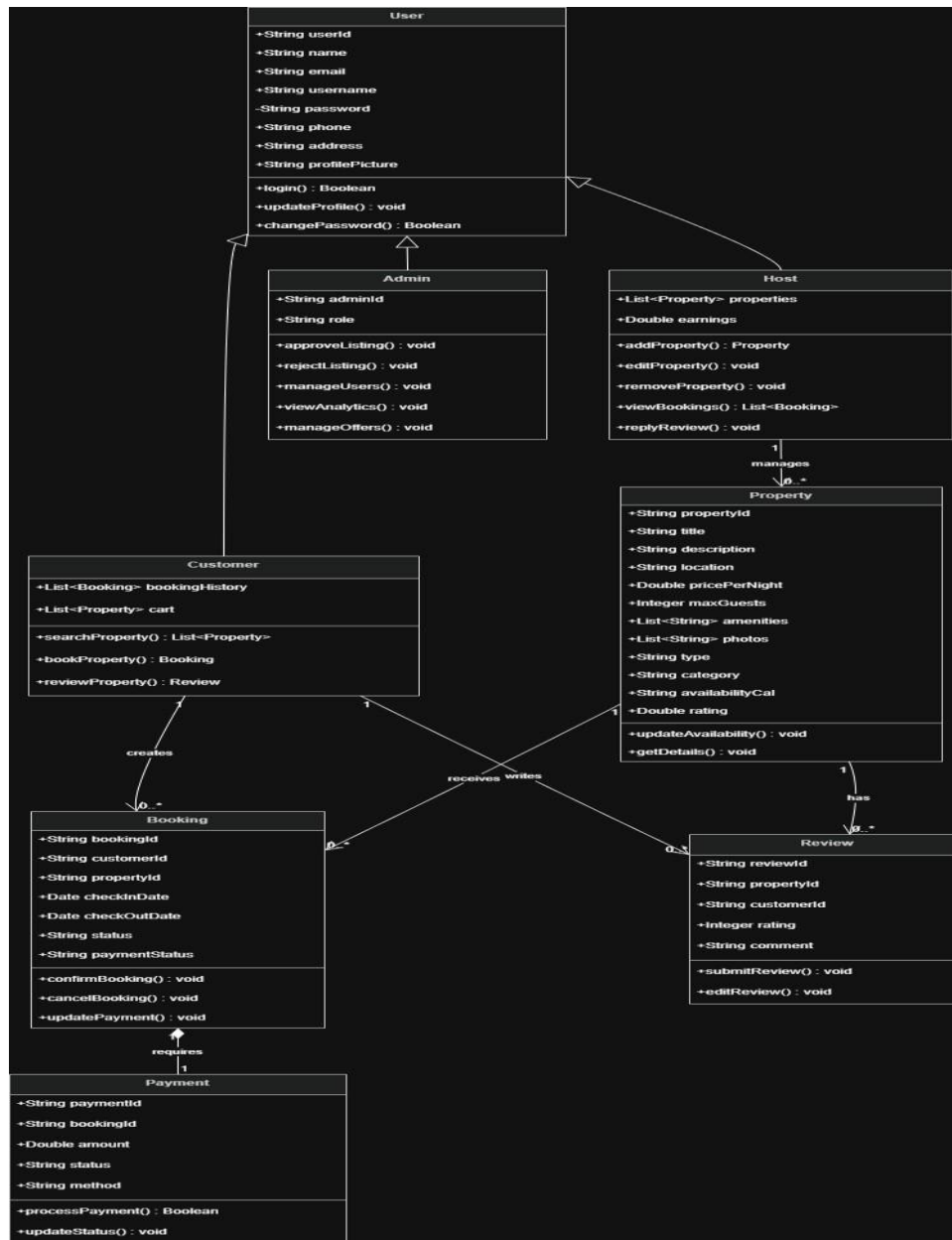
4. SOFTWARE DESIGN

4.1 System Design:

Use case Diagram



Class diagram:



3.1 UI / Wireframe Design using Figma

Search Home:



Properties in your area

Filters

Price Range

Min

Max

Property Type

☒ Single Family Home☐ Condo / Apartment☐ Townhouse

Amenities

Pool

Gym

Parking

Garden

Play area

Apply Filters

Clear all



Azure Horizon

\$850

60 Heritage Way, Silver Springs

Stunning contemporary architecture featuring open-concept living, smart home integration, and energy-efficient systems throughout.

3 Bed 3 Bath

View Details



Ocean Breeze Suite

\$620,000

88 Marine View, Say Harbor

Luxury apartment with panoramic ocean views, private balcony access, and access to a premium rooftop wellness center.

2 Bed 2 Bath

View Details



Willow Creek Cottage

\$495,000

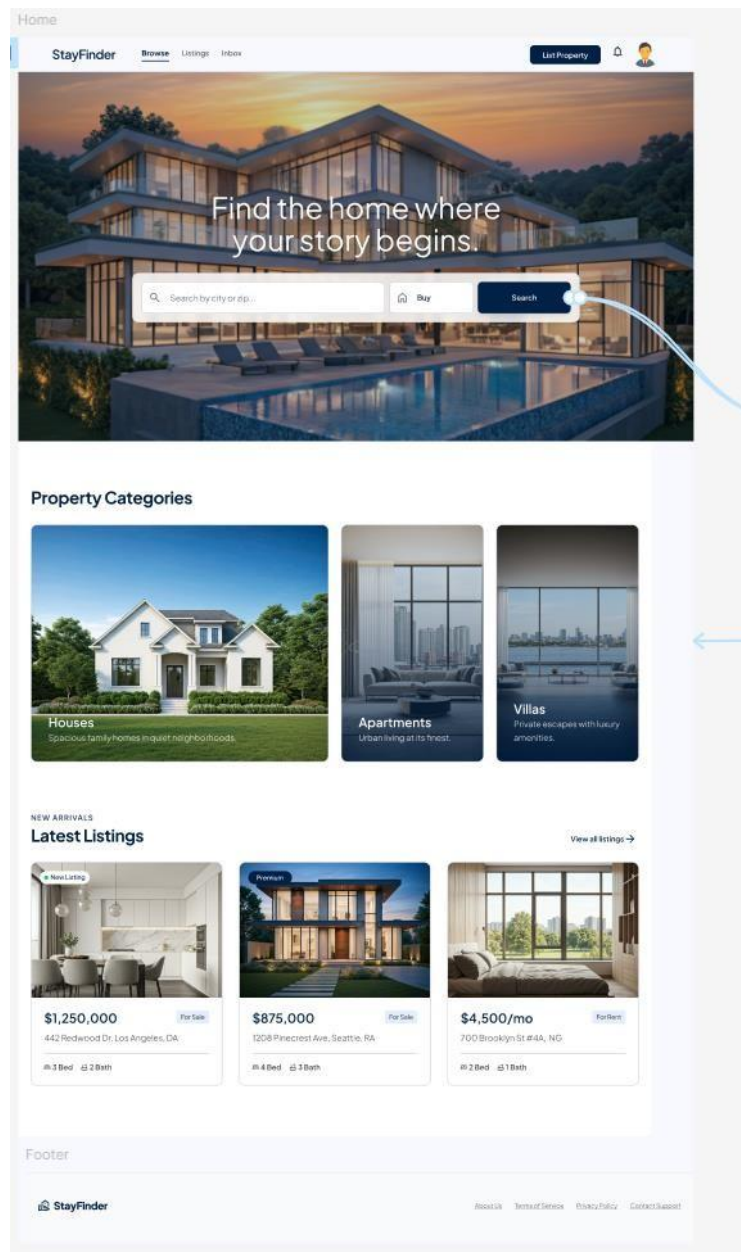
45 Old Oak Road, Fairfield

A beautifully renovated traditional home that combines timeless charm with modern comforts, featuring a spacious garden and an elegant wrap-around porch.

3 Bed 2 Bath

View Details

Home:



Add property:

AddProperty

StayFinder

Browse Listings Inbox Account

Property Basics

Property Title
e.g. Modern Villa with Sunset View

Category: Villa Price per Night (USD): \$ 150

Location

City & Country
e.g. Dhaka

Upload Photos

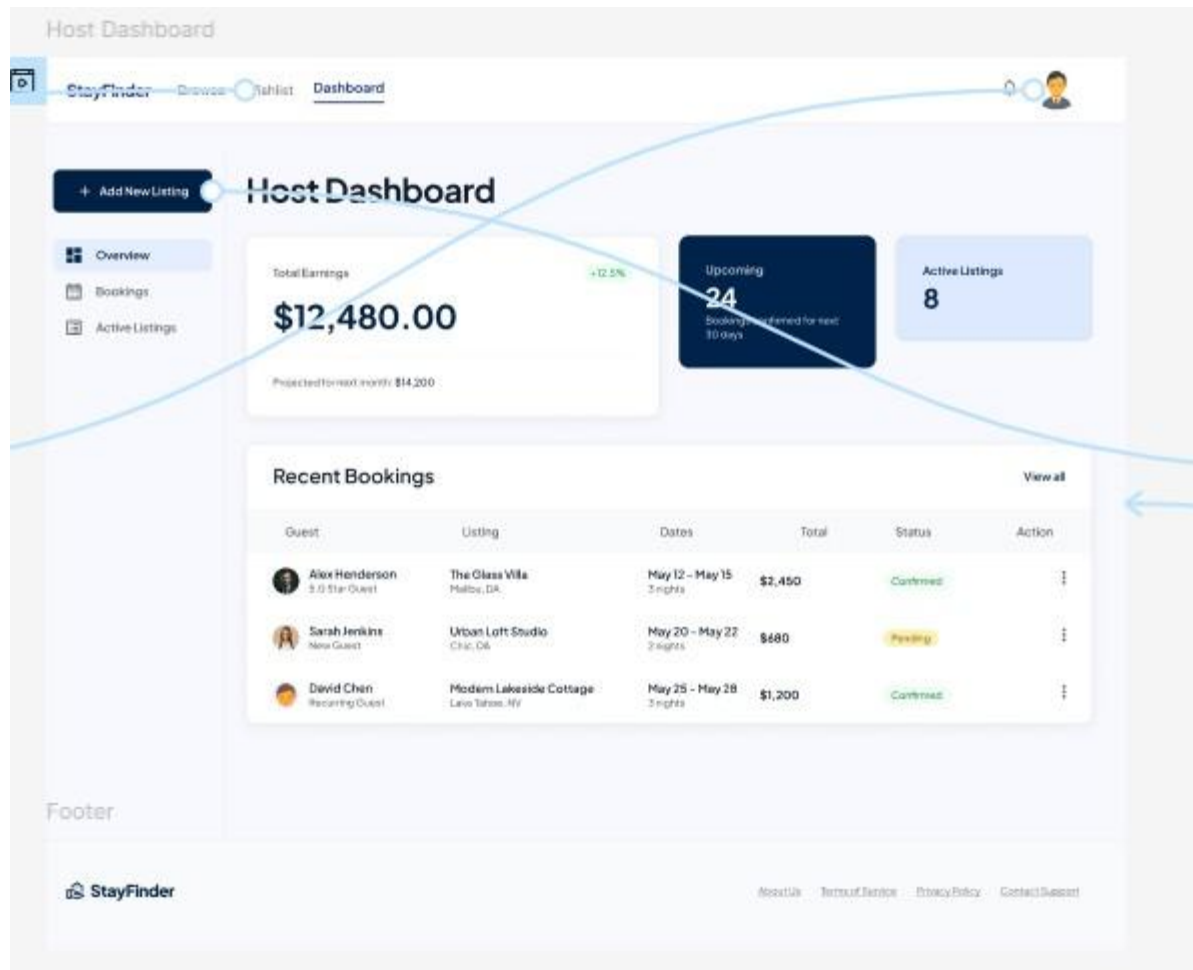
Click to upload or drag and drop

Save as Draft Post Listing

Footer

StayFinder About Us Terms of Service Privacy Policy Contact Us/Help

Host Dashboard:



Profile Setting:

Profile Settings

StayFinder

Browse

Wishlist

Dashboard

Sign In

Personal Info

Security

Payments

Notifications

Profile Settings

Manage your personal information and how we reach you.

Your Photo

Upload a high-resolution image for better profile visibility.

Upload New

Remove

Full Name

Tashin

Email Address

Tashin@example.com

Phone Number

+18825245

Residential Address

123 Serenity Drive

Discard Changes

Save Changes

StayFinder

About Us

Terms of Service

Privacy Policy

Contact Us/Support

Booking History:

Page 55 of 112

Booking History

StayFinder

BrowseListingsInbox

Account

Your Booking History

Manage and review all your past property viewings and stays.

Dashboard

Booking History

Saved Properties

Account Settings

PROPERTY NAME	DATES	TOTAL PAID	STATUS	
 The Glass Pavilion Skyline Heights, GA	Oct 12 - Oct 15, 2024 3 Nights	\$1,250.00	Completed	View Details
 Oakwood Manor Greenland Estates, OR	Sep 20 - Sep 22, 2024 2 Nights	\$540.00	Cancelled	Rebook
 Pine Needle Retreat Aspen Creek, CO	Aug 05 - Aug 10, 2024 5 Nights	\$2,100.00	Completed	View Details
 Loomis Lofts #402 Downtown Seattle, WA	Jul 14 - Jul 15, 2024 1 Night	\$235.00	Completed	View Details

12

Footer

StayFinder

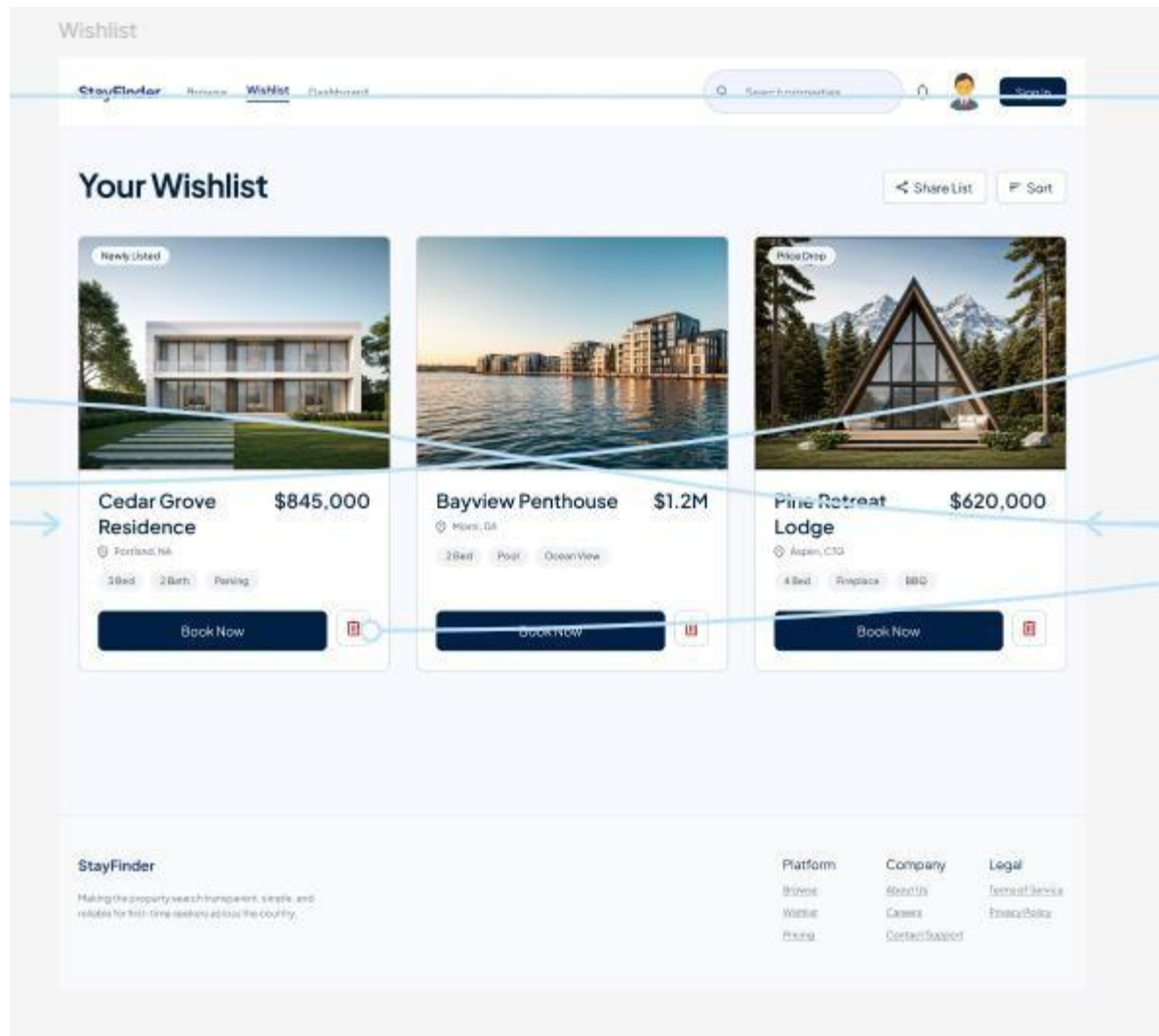
About Us

Terms of Service

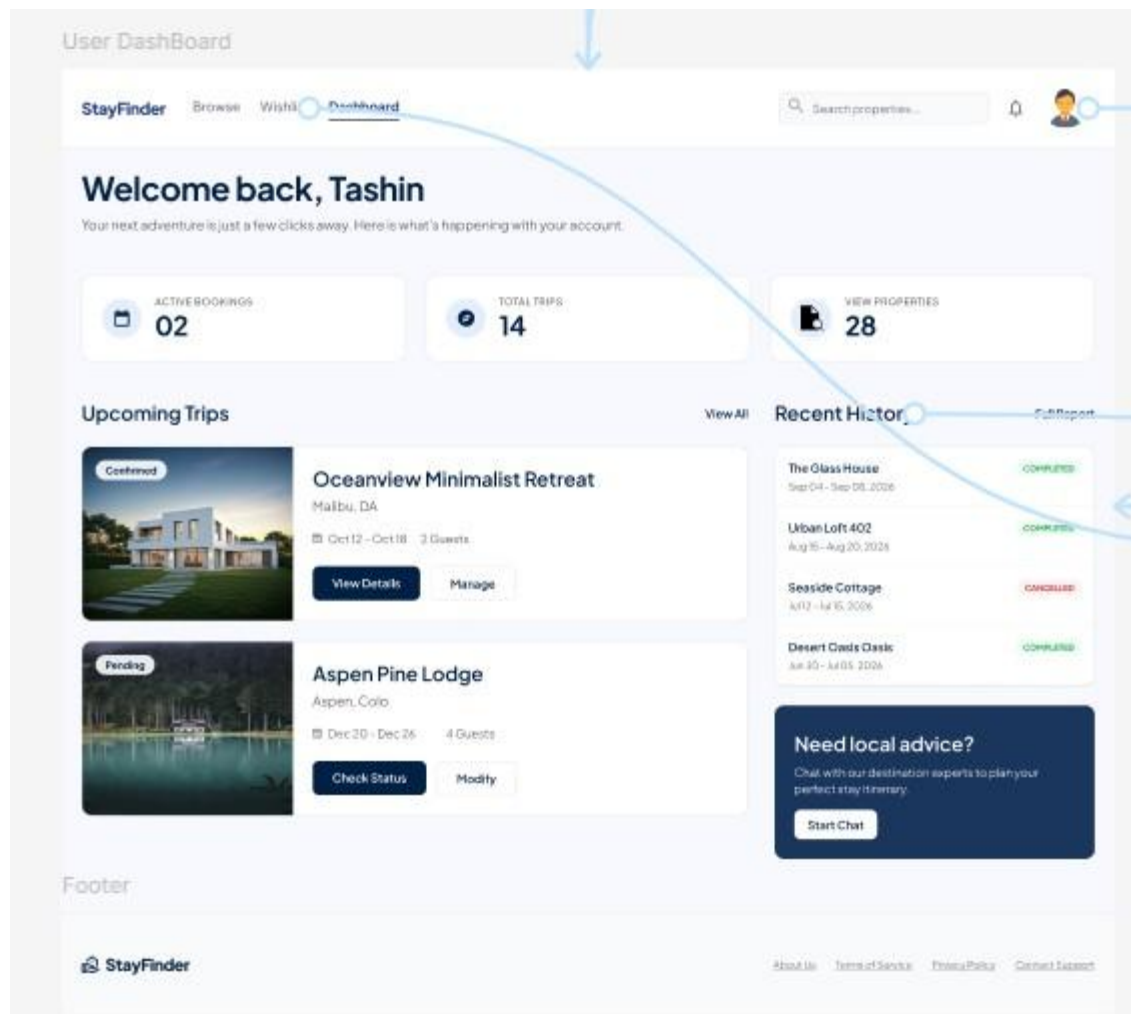
Privacy Policy

Contact Support

Wishlist:



User Dashboard:



Admin Panel:

Admin Panel

StayFinder Browse Wishlist Dashboard

Admin Control Center

Real-time system

Total Users

12,842

+12% this month

Total Revenue

\$412,050

+5.4% this month

Reported Listings

24

† Requires immediate action

User Management

Search users...

User	Role	Status	Actions
 Jane Doe jane@sample.com	Agent	ACTIVE	
 Mark Smith mark.s@property.co	Buyer	ACTIVE	
 Alice Luna alice.l@web.com	Agent	SUSPENDED	

[View All 12,842 Users](#)

Moderation Queue

34



Modern Villa in suburbs
Reported for: Duplicate Listing



Penthouse Downtown
Reported for: Misleading Price



Countryside Retreat
Reported for: Spam

[Go to full Queue](#)

Footer

 StayFinder

[About Us](#) [Terms of Service](#) [Privacy Policy](#) [Contact Support](#)

Payment:

Payment

StayFinder

CHECKOUT

Payment Method

Card

PayPal

Transfer

Card Number

0000 0000 0000 0000

Expiration Date

MM / YY

CVV

123

Cardholder Name

Full name as it appears on card

Billing Address

☒ Same as shipping

Street Address

123 Residential Way

City

Region

Postal Code

Bangladesh

South Asia

5400

Summary

Security Deposit

\$2,450.00

Service Fee

\$45.00

Total Amount

\$2,49.00

Complete Payment

Verified Secure Transaction

Your personal data and transaction details are protected by industry-standard 256-bit SSL encryption

StayFinder

About Us

Terms of Service

Privacy Policy

Contact Support

Confirm Booking:

Confirm Booking

StayFinder

[Browse](#)[Wishlist](#)[Dashboard](#)



[← Back to property](#)

Confirm and pay

Your trip

Dates

May 12 - May 17, 2026

Edit

Guests

2 guests

Edit



ENTIRE VILLA

The Glass Pavilion at Azure Heights

★ 4.92

Ground rules

 Follow the house rules

 Treat your Host's home like your own

Cancellation policy

Free cancellation before May 5. After that, the reservation is non-refundable.

Make sure this Host's cancellation policy works for you. Our Guest Refund Policy still applies.

Price details

\$450.00 x 5 nights	\$2,250.00
Cleaning fee	\$120.00
Estate Guide service fee	\$315.00
Taxes	\$85.00
Total (USD)	\$2,770.00

Confirm and Pay

You won't be charged yet

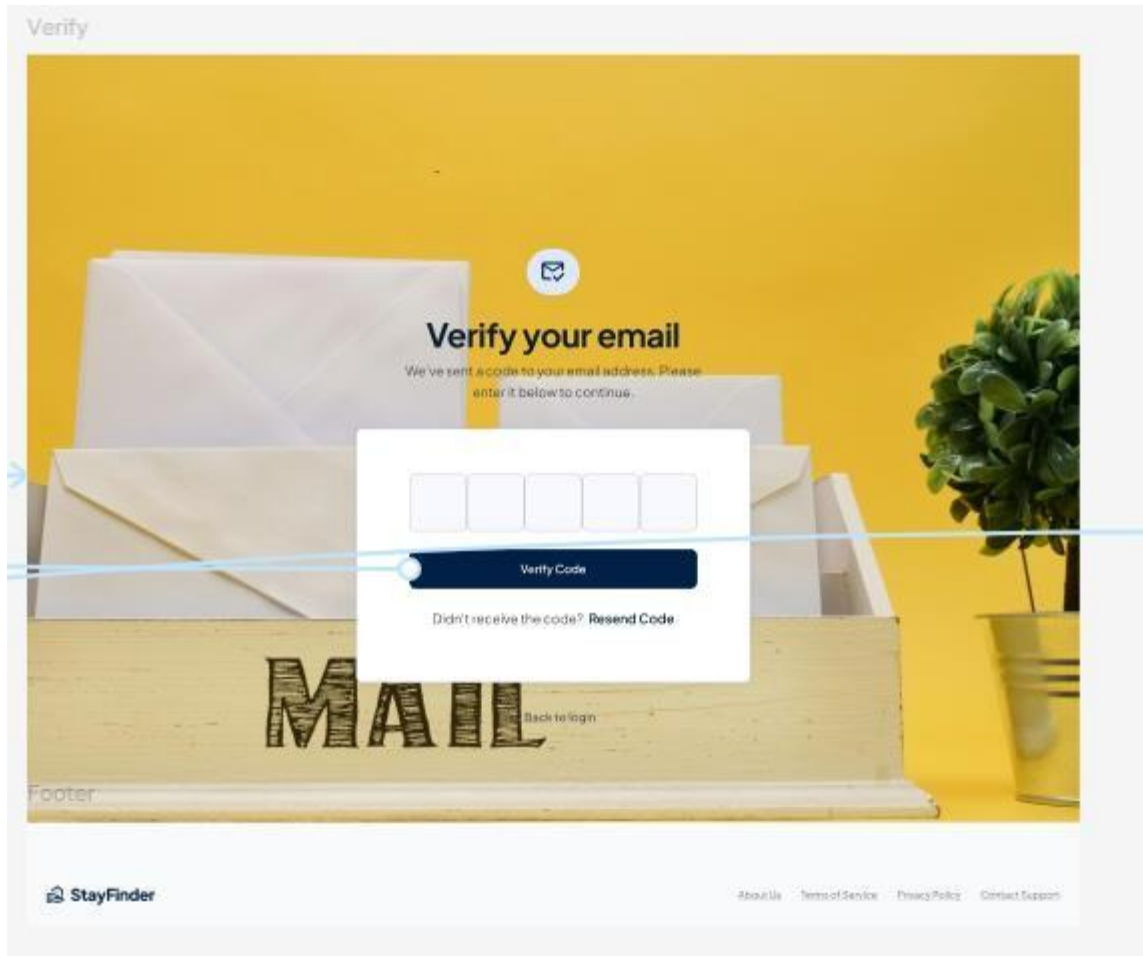
Footer

 StayFinder

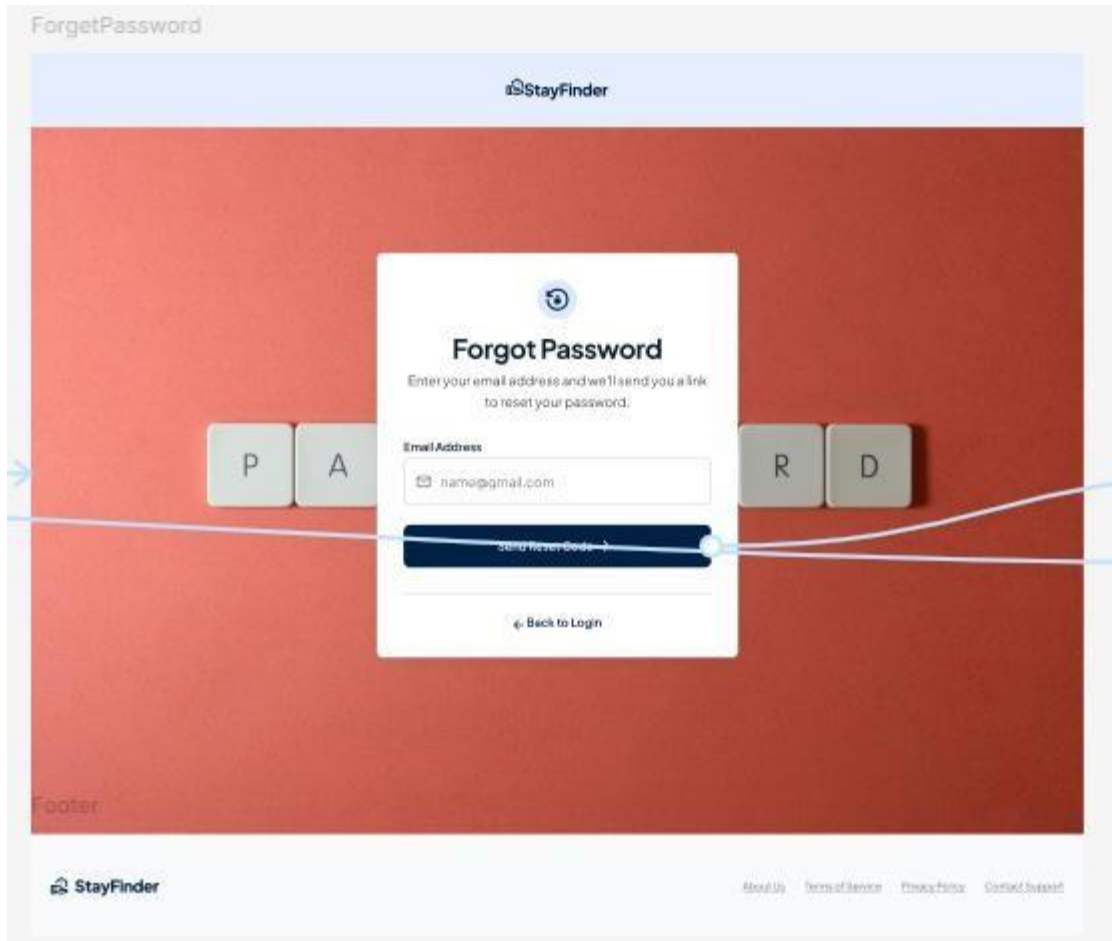
[About Us](#)[Contact Us](#)[Privacy Policy](#)[Contact Support](#)

Verify:

Page 61 of 112

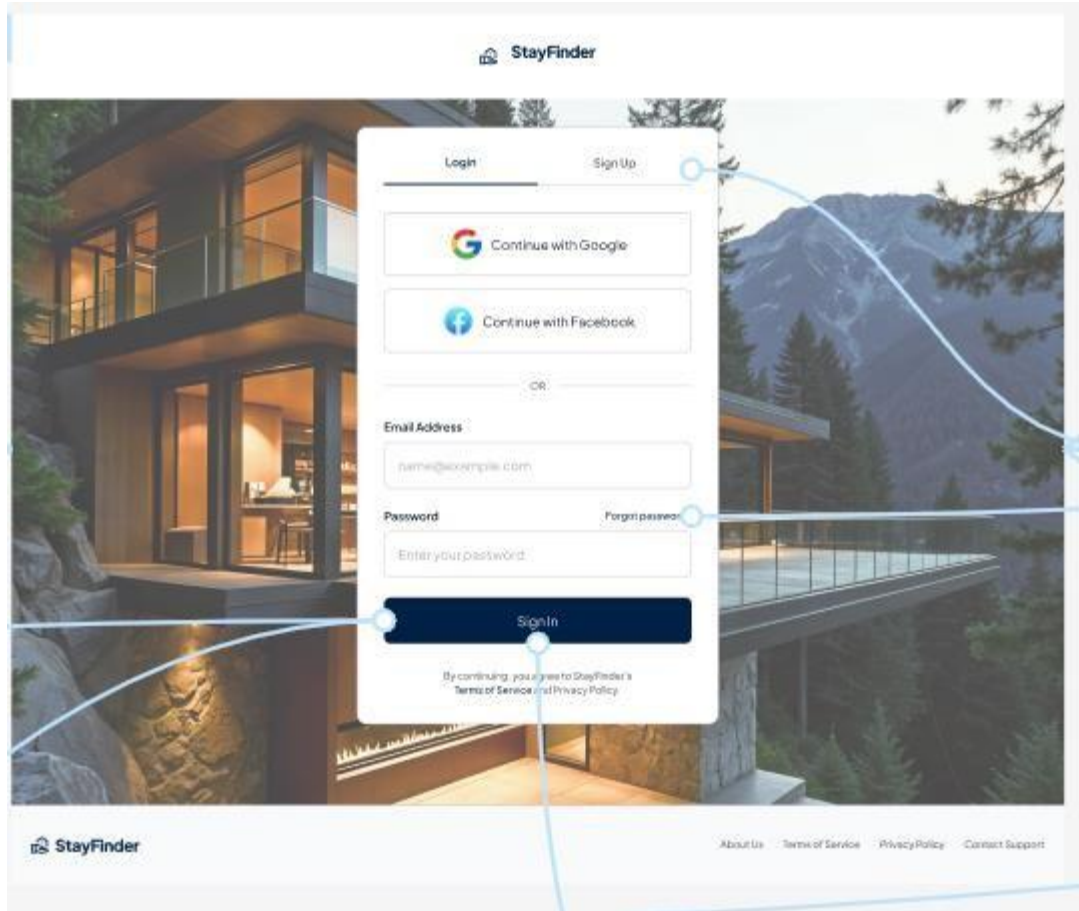


Forget Password:

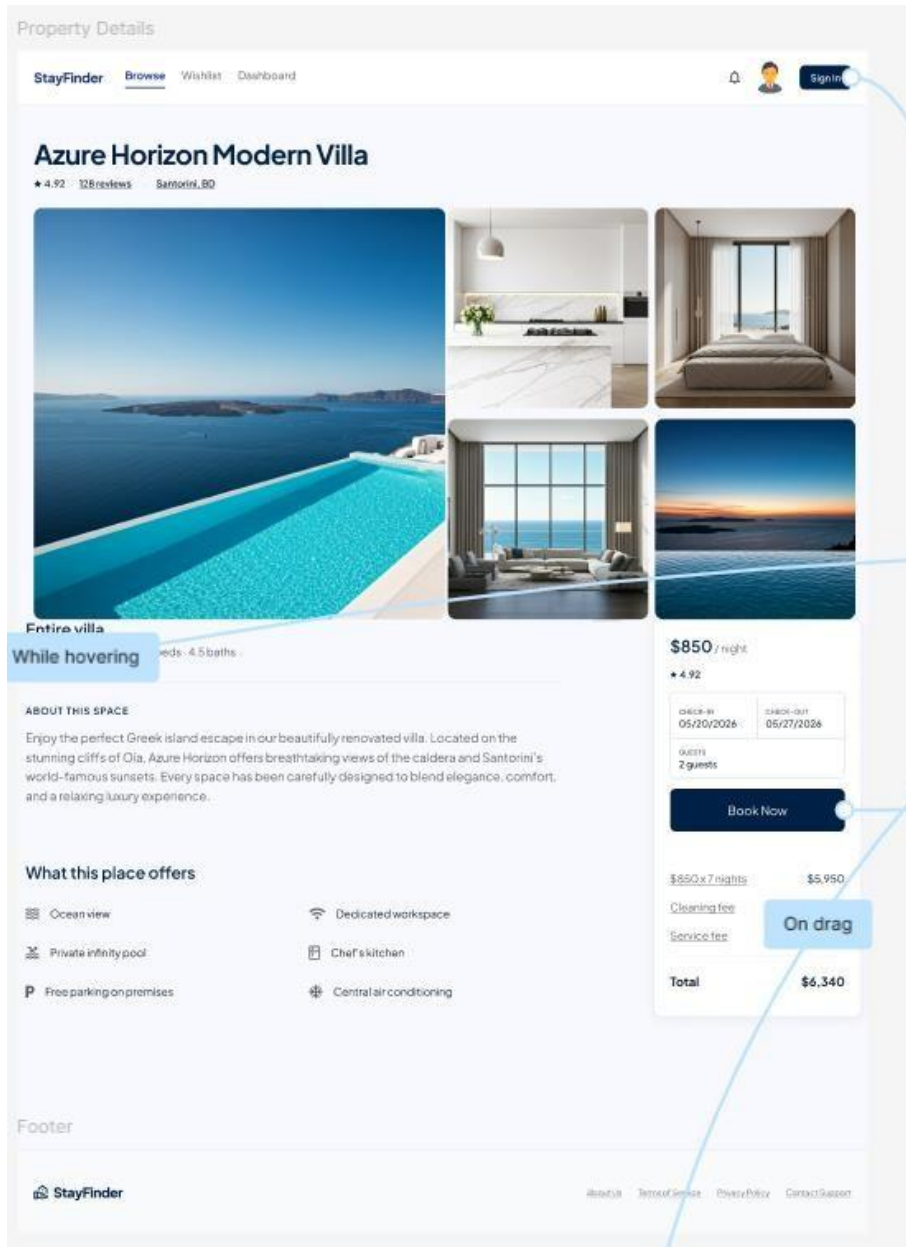


Sign Up:

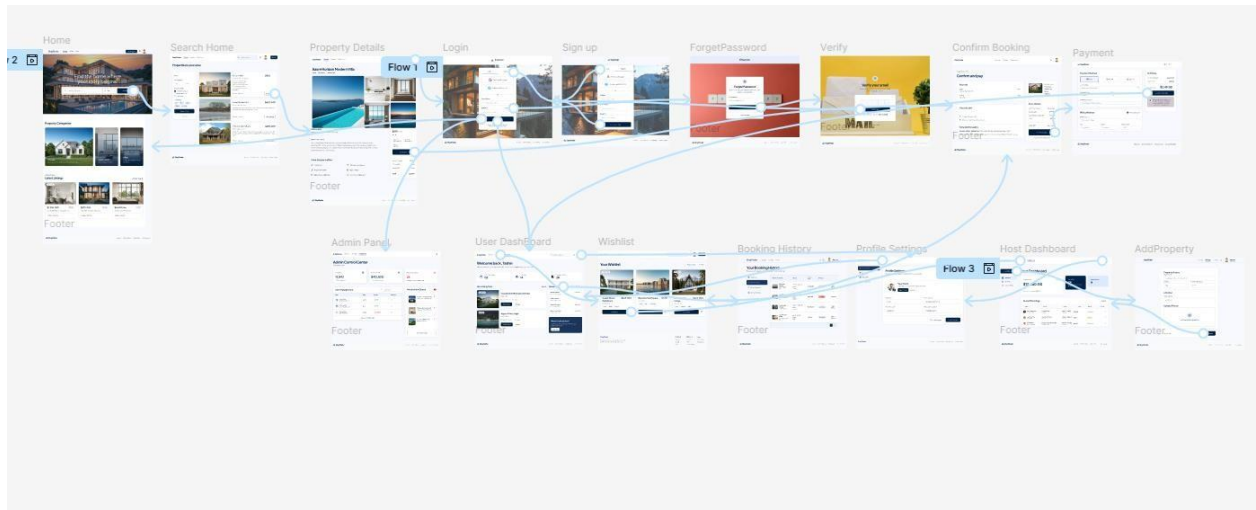




Property Details:



Figma Prototype:



Figma link:

<https://www.figma.com/design/YKsKFPxQXpB4jZAmev1jvP/StayFinder?node-id=23469&p=f&t=ifp2dWZIYjOB6ueM-0>

5. GIT WORKFLOW

main 17 Branches 0 Tags

Go to file

Add file

Code

Tashin90 Merge pull request #13 from Tashin90/feature/FR-customer-booking-history 34a09af · 11 minutes ago 39 Commits

CHANGELOG.md	chore: initial project setup and documentation	2 hours ago
PROJECT_FEATURES.md	Merge branch 'main' into feature/FR-customer-booking-hist...	11 minutes ago
README.md	Update README.md	1 hour ago

README



Project Simulator

#software

```

remote:      https://github.com/Tashin90/StayFinder/pull/new/feature/FR-admin-property-approval
remote:
To https://github.com/Tashin90/StayFinder.git
 * [new branch]      feature/FR-admin-property-approval -> feature/FR-admin-property-approval
branch 'feature/FR-admin-property-approval' set up to track 'origin/feature/FR-admin-property-approval'.

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-property-approval)
$ git checkout dev
Switched to branch 'dev'
Your branch is ahead of 'origin/dev' by 2 commits.
  (use "git push" to publish your local commits)

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (dev)
$ git pull origin dev
From https://github.com/Tashin90/StayFinder
 * branch            dev                -> FETCH_HEAD
Already up to date.

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (dev)
$ git checkout -b feature/FR-admin-category-management
Switched to a new branch 'feature/FR-admin-category-management'

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "## FR-admin-category-management: Property Category Management System" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "The system allows administrators to manage property categories and types." >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "- Add category functionality enabled" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "- Edit and delete category options added" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "- Property type management system added" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "- Category validation implemented" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ echo "***Status: Implemented***" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ git add PROJECT_FEATURES.md
warning: in the working copy of 'PROJECT_FEATURES.md', LF will be replaced by CRLF the next time Git touches it

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$
git commit -m "feat: implement property category management system (FR-admin-category-management)"

- Added category management functionality
- Added edit and delete options
- Added property type management
- Added validation for categories"
[feature/FR-admin-category-management 66b16df] feat: implement property category management system (FR-admin-category-management)
1 file changed, 8 insertions(+)

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ git push -u origin feature/FR-admin-category-management
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 631 bytes | 631.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/FR-admin-category-management' on GitHub by visiting:
remote:      https://github.com/Tashin90/StayFinder/pull/new/feature/FR-admin-category-management
remote:
To https://github.com/Tashin90/StayFinder.git
 * [new branch]      feature/FR-admin-category-management -> feature/FR-admin-category-management
branch 'feature/FR-admin-category-management' set up to track 'origin/feature/FR-admin-category-management'.

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-admin-category-management)
$ |

```

```

$ git checkout -b feature/FR-host-add-property
Switched to a new branch 'feature/FR-host-add-property'

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "## FR-host-add-property: Add Property Listing System" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "The system allows hosts to create and publish property listings." >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "- Property title and description required" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "- Image upload functionality added" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "- Price and location validation added" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "- Property publish option enabled" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ echo "**Status: Implemented**" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ git add PROJECT_FEATURES.md
warning: in the working copy of 'PROJECT_FEATURES.md', LF will be replaced by CRLF the next time Git touches it

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ git commit -m "Feat: implement host property listing system (FR-host-add-property)"

- Added property upload form
- Added image upload support
- Added validation for property details
- Added publish listing functionality"
[feature/FR-host-add-property 3e0895a] feat: implement host property listing system (FR-host-add-property)
1 file changed, 8 insertions(+)

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ git push -u origin feature/FR-host-add-property
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 621 bytes | 621.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/FR-host-add-property' on GitHub by visiting:
remote:   https://github.com/Tashin90/StayFinder/pull/new/feature/FR-host-add-property
remote:
To https://github.com/Tashin90/StayFinder.git
 * [new branch]      feature/FR-host-add-property -> feature/FR-host-add-property
branch 'feature/FR-host-add-property' set up to track 'origin/feature/FR-host-add-property'.

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-add-property)
$ git checkout dev
Switched to branch 'dev'
Your branch is ahead of 'origin/dev' by 2 commits.
  (use "git push" to publish your local commits)

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (dev)
$ git pull origin dev
From https://github.com/Tashin90/StayFinder
 * branch      dev      -> FETCH_HEAD
Already up to date.

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (dev)
$ git checkout -b feature/FR-host-manage-property
Switched to a new branch 'feature/FR-host-manage-property'

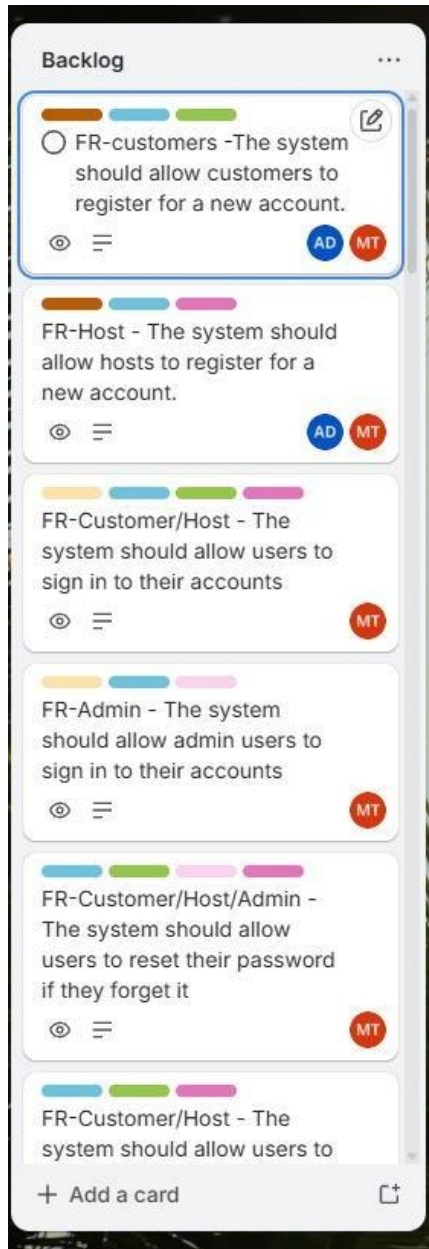
Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-manage-property)
$ echo "" >> PROJECT_FEATURES.md

Tashin@LAPTOP-GOULG5VC MINGW64 /c/StayFinder (feature/FR-host-manage-property)
$ echo "## FR-host-manage-property: Manage Property Listings System" >> PROJECT_FEATURES.md

```

Github link: <https://github.com/Tashin90/StayFinder>

Trello Board:



Trello link:

<https://trello.com/b/BqPp79V2/stayfinder>

6. SOFTWARE TESTING

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Abdullah Al Mahbub
Test Case ID: TC_AUTH_01		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	
Module Name: Customer Registration Module		Test Execution date:	
Test Title: Verify that a customer can register for a new account			
Description: Ensure the system allows a customer to register by filling all required fields, validates email uniqueness, enforces password rules, sends OTP, and redirects to the Customer Dashboard upon successful registration.			
Precondition: User is on the Customer Registration page. No existing account with the test email address.			
Dependencies: OTP/Email delivery service must be operational; database must be accessible.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)

1. Open the Customer Registration page and observe the form fields	N/A	Registration form loads with fields: Full Name, Email, Username, Password, Confirm Password, Phone Number, Address, and a 'Register' button		
2. Fill all required fields with valid data and click 'Register'	Full Name: John Doe, Email: john@test.com , Username: johndoe, Password: John@1234, Phone: 01700000001, Address: Dhaka	OTP is sent to the registered phone number; system prompts for OTP entry to complete verification		
3. Enter a valid OTP to complete verification	Valid OTP received on phone	OTP is verified successfully; a confirmation email is sent; user is redirected to the Customer Dashboard		
4. Enter an already registered email in the Email field and submit	Email: john@test.com (already used)	System displays 'Invalid or already used email' message; form submission is blocked		
5. Enter a password with fewer than 8 characters and submit	Password: Jo@1	System shows validation error: password must be minimum 8 characters long		
6. Enter a Confirm Password that does not match the Password field and submit	Password: John@1234, Confirm Password: John@5678	System shows 'Password do not match' message; form submission is blocked		

7. Submit the registration form leaving the 'Full Name' field empty	Full Name: empty; all other fields filled	System highlights the empty required field and prevents form submission		
8. Verify the page footer contains Terms & Conditions and Privacy Policy links	Observe page footer	Both Terms & Conditions and Privacy Policy links are visible and clickable in the footer		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Abdullah Al Mahbub
Test Case ID: TC_AUTH_02		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	
Module Name: Host Registration Module		Test Execution date:	
Test Title: Verify that a host can register for a new account			
Description: Ensure the system allows a host to register with all mandatory fields, enforces password complexity rules, verifies identity via OTP, displays a success message, and redirects to the Host Dashboard.			
Precondition: User is on the Host Registration page. No existing account with the test email address.			

Dependencies: OTP/Email verification service must be active; database must be accessible.

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Open the Host Registration page and observe form fields	N/A	Form loads with all required fields: Full Name/Business Name, Email, Username, Password, Confirm Password, Phone Number, Business Address, and a 'Register as Host' button		
2. Submit the form leaving a mandatory field empty	Leave 'Business Address' blank; all other fields filled	System shows appropriate error message for the missing required field; form is not submitted		
3. Enter a password without a special character and submit	Password: HostPass1	System shows error: password must meet complexity rules (minimum length and at least one special character)		
4. Fill all mandatory fields with valid data and click 'Register as Host'	Business Name: Stay Villa, Email: host@test.com , Password: Host@1234, Phone: 01800000001	OTP is sent for email/phone verification; system prompts for OTP entry		
5. Enter a valid OTP to complete verification	Valid OTP received	Verification succeeds; success message is displayed; user is redirected to the Host Dashboard		

6. Enter a duplicate email already registered on the platform and submit	Email: host@test.com (already registered)	System shows error for duplicate/invalid input; form is not submitted		
7. Verify the Host Dashboard is fully accessible after successful registration	Check page after redirect	Host Dashboard loads correctly with host name and relevant sections visible		
8. Verify the registration page footer contains Terms & Conditions and Privacy Policy links	Observe page footer	Both links are present and navigable in the page footer		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Abdullah Al Mahbub
Test Case ID: TC_AUTH_03		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	
Module Name: User Sign In Module		Test Execution date:	
Test Title: Verify that users can sign in to their accounts			

Description: Ensure the sign-in page authenticates customers and hosts, redirects each to the correct dashboard, shows error on invalid credentials, and supports Google Sign-In and Remember Me session persistence.

Precondition: At least one active Customer account and one active Host account exist. User is on the Sign In page.

Dependencies: Database must have registered users; Google OAuth must be configured.

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Enter valid Customer credentials and click 'Sign In'	Email: customer@test.com , Password: Cust@1234	User is redirected to the Customer Dashboard; session is established successfully		
2. Enter valid Host credentials and click 'Sign In'	Email: host@test.com , Password: Host@1234	User is redirected to the Host Dashboard; session is established successfully		
3. Enter an incorrect password for a valid email and click 'Sign In'	Email: customer@test.com , Password: WrongPass	System shows 'Invalid email or password' message; no dashboard redirect occurs		
4. Enable 'Remember Me', sign in with valid credentials, close and reopen the browser	Remember Me: checked; Email: customer@test.com , Password: Cust@1234	User session is retained for 30 days; user remains signed in after reopening the browser		

5. Click the 'Sign In with Google' option and complete Google authentication	Valid Google account	Google OAuth popup opens; after successful auth, user is redirected to the appropriate dashboard		
6. Leave the Email field blank and click 'Sign In'	Email: empty, Password: Cust@1234	System shows a validation error for the empty email field; form is not submitted		
7. Verify all required elements are present on the Sign In page	Observe Sign In page	Page shows Email field, Password field, Remember Me checkbox, Google Sign In button, and Forgot Password link		
8. Click the 'Forgot Password' link	N/A	User is navigated to the Password Recovery page		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Abdullah Al Mahbub
Test Case ID: TC_AUTH_04		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	
Module Name: Admin Sign In Module		Test Execution date:	

Test Title: Verify that admin users can sign in to their accounts
Description: Ensure the Admin Sign In page validates @stayfinder.com email domain, matches the password, redirects to Admin Dashboard on success, and locks the account for 15 minutes after 5 consecutive failed attempts.
Precondition: An Admin account with a valid @stayfinder.com email exists. User is on the Admin Sign In page.
Dependencies: Admin accounts must exist in the database; account lockout mechanism must be enabled.

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Enter an email not ending with @stayfinder.com and click 'Sign In'	Email: admin@gmail.com , Password: Admin@1234	System shows 'Invalid admin email' message; login is blocked		
2. Enter a valid @stayfinder.com email with an incorrect password and click 'Sign In'	Email: admin@stayfinder.com , Password: WrongPass	System shows 'Invalid email or password' message; no dashboard access is granted		
3. Fail login 5 consecutive times with wrong credentials	5 incorrect password attempts for admin@stayfinder.com	Account is locked after the 5th failed attempt; system displays an account lockout message		
4. Attempt to sign in immediately after the account is locked	admin@stayfinder.com with any password within 15-minute lockout period	System blocks the login attempt and informs user the account is locked for 15 minutes		

5. Wait 15 minutes and sign in with valid admin credentials	Email: admin@stayfinder.com , Password: Admin@1234 (after lockout expires)	Login succeeds; admin is redirected to the Admin Dashboard		
6. Enter valid admin credentials and click 'Sign In'	Email: admin@stayfinder.com , Password: Admin@1234	Admin is successfully redirected to the Admin Dashboard; session is established		
7. Verify the Admin Sign In page has a 'Forgot Password' option	Observe the Admin Sign In page	'Forgot Password' link is visible and navigable on the page		
8. Verify the Admin Dashboard loads completely after successful login	Valid admin credentials	Admin Dashboard renders fully with admin navigation and management sections visible		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Abdullah Al Mahbub
Test Case ID: TC_AUTH_05		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	

Module Name: Password Reset Module		Test Execution date:	
Test Title: Verify that users can reset their password if they forget it			
Description: Ensure the password reset flow validates the registered email, delivers a time-limited OTP (10 minutes), enforces new password rules (min 8 chars, special character), confirms password match, and redirects to Sign In on success.			
Precondition: A registered user account exists. User is on the Forgot Password / Password Recovery page.			
Dependencies: Email OTP delivery service must be functional.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Enter an unregistered email and click 'Send OTP'	Email: noone@test.com	System shows 'No account found' message; OTP is not sent		
2. Enter a valid registered email and click 'Send OTP'	Email: customer@test.com	OTP is sent to the registered email address; OTP input field appears on screen		
3. Enter the received OTP within 10 minutes and proceed	Valid OTP from email	OTP is accepted; user is allowed to set a new password		
4. Enter an OTP after the 10-minute validity window has expired	Expired OTP	System shows OTP expired message; user must request a new OTP to proceed		

5. Enter a new password shorter than 8 characters	New Password: Pass1!	System shows error: new password must be minimum 8 characters long		
6. Enter a new password without any special character	New Password: Password1	System shows error: new password must contain at least one special character		
7. Enter a mismatching Confirm Password and attempt to reset	New Password: NewPass@1, Confirm Password: NewPass@2	System shows 'Password do not match' message; password reset is blocked		
8. Complete password reset with all valid inputs	Valid OTP, New Password: NewPass@1, Confirm Password: NewPass@1	System shows 'Password reset successful' message and redirects user to the Sign In page		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Fariha Mahafuz Medha
Test Case ID: TC_AUTH_06		Test Designed date:	17-May-2026
Test Priority: Medium		Test Executed by:	
Module Name: User Profile Management Module		Test Execution date:	

Test Title: Verify that users can view and edit their profile information
Description: Ensure the profile page pre-fills all existing fields, validates email format and phone number length, restricts photo uploads to PNG/JPG within 2MB, verifies old password on change, and shows a success notification upon saving.
Precondition: User is logged in as Customer or Host and has navigated to the Profile page.
Dependencies: User profile data must exist in the database.

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Profile page and observe all fields	N/A	All existing profile fields (Full Name, Email, Phone, Date of Birth, Address, Profile Photo) are pre-filled with current data		
2. Edit the Full Name field and click 'Save Changes'	Full Name: Jane Doe	Success notification is displayed; profile page is refreshed showing the updated name		
3. Enter an invalid email format and click 'Save Changes'	Email: janedoe#test	System shows email format validation error; changes are not saved		
4. Enter a phone number with fewer than 11 digits and click 'Save Changes'	Phone: 01700	System shows phone number validation error indicating 11 digits are required		
5. Upload a profile photo in PNG format within 2MB and save	File: photo.png, Size: 1.5MB	Photo is uploaded and displayed on the profile; success notification is shown		

6. Attempt to upload a profile photo larger than 2MB	File: bigphoto.jpg, Size: 5MB	System shows error: maximum file size is 2MB; the photo is rejected and not uploaded		
7. Attempt to change password with an incorrect old password	Old Password: WrongOld, New Password: NewPass@1	System shows error: old password is incorrect; password is not updated		
8. Attempt to upload a profile photo in an unsupported format	File: image.gif	System shows error: only PNG/JPG formats are accepted; the file is not uploaded		
9. Change password with correct old password and a valid new password	Old Password: Cust@1234, New Password: NewPass@1 (8+ chars)	Password is updated successfully; success notification is displayed		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Fariha Mahafuz Medha
Test Case ID: TC_PROP_01		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	

Module Name: Property Listing Management Module		Test Execution date:	
Test Title: Verify that hosts can add new property listings			
Description: Ensure a host can submit a property listing with all required fields and at least one valid photo, that price must be a positive number, and the listing is saved with 'Pending Approval' status with host notification on admin action.			
Precondition: Host is logged in and has navigated to the Add Property Listing page.			
Dependencies: Host account must be active; admin approval workflow must be in place.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Fill all required fields and upload one valid photo, then click 'Submit Listing'	Title: Sea View Villa, Price: 5000, Location: Cox's Bazar, Type: Villa, Category: Luxury, Photo: room.jpg (3MB)	Listing is saved with status 'Pending Approval'; host sees a confirmation message		
2. Submit listing without uploading any photo	All required fields filled; no photo uploaded	System shows error: minimum 1 photo is required; listing is not submitted		
3. Upload a photo exceeding 5MB	Photo: largeroom.jpg, Size: 8MB	System shows error: each photo must not exceed 5MB; the photo is rejected		

4. Enter a negative value in the Price field and submit	Price: -500	System shows validation error: price must be a positive number; listing is not submitted		
5. Verify listing is not visible in customer search before admin approval	Search for 'Sea View Villa' as a Customer	Listing is not visible in customer-facing search results while status is 'Pending Approval'		
6. Have admin approve the listing and verify host receives a notification	Admin approves listing from Admin Panel	Host receives a notification (email/in-app) confirming that the listing has been approved		
7. Have admin reject the listing and verify host receives notification with reason	Admin rejects listing with reason: 'Incomplete images'	Host receives a notification with the rejection reason provided by the admin		
8. Verify the listing form contains all required fields including Amenities and Availability	Observe listing creation form	Form includes: Property Title, Description, Type, Category, Price Per Night, Location, Photos, Amenities, and Availability fields		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Fariha Mahafuz Medha
Test Case ID: TC_PROP_02		Test Designed date:	17-May-2026

Test Priority: High		Test Executed by:	
Module Name: Property Listing Edit & Removal Module		Test Execution date:	
Test Title: Verify that hosts can edit or remove their property listings			
Description: Ensure the host can view all listings with status, edit listing data (resetting status to Pending Approval), see a confirmation before removal, and have removed listings disappear from customer search immediately.			
Precondition: Host is logged in with at least one existing property listing. Host is on the Property Management page.			
Dependencies: Active and pending property listings must exist for the host.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Property Management page and observe all listings	N/A	All host listings are displayed with their current status: Active, Pending, or Rejected		
2. Click 'Edit' on an active listing, update the Title, and click 'Save Changes'	New Title: Updated Sea View Villa	Edit form is pre-filled with existing data; after saving, listing status changes to 'Pending Approval' and a success message is shown		
3. Verify that editing a listing resets its status to 'Pending Approval'	Edit any active listing	After saving edits, listing status becomes 'Pending Approval' — not Active		

4. Click 'Remove Listing' for a selected listing	Any existing listing	Confirmation dialog appears before removal action is executed		
5. Confirm the removal in the confirmation dialog	Click 'Confirm' on the dialog	Listing is removed; it no longer appears in customer		
		property search results immediately		
6. Verify the edit form is pre-filled with all existing data	Open edit form for any listing	All fields — Title, Description, Type, Category, Price, Location, Photos, Amenities — are pre-populated with current data		
7. Cancel the removal confirmation dialog	Click 'Cancel' on the confirmation dialog	Listing is not removed; it remains in the host's listing table unchanged		
8. Verify a rejected listing is visible in the host's listing table with correct status	Check rejected listing in Property Management	Rejected listing appears with status 'Rejected' in the host's Property Management page		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Fariha Mahafuz Medha
Test Case ID: TC_PROP_03		Test Designed date:	17-May-2026

Test Priority: High		Test Executed by:	
Module Name: Property Approval Module		Test Execution date:	
Test Title: Verify that administrators can approve or reject property listings			
Description: Ensure the admin can view all pending listings, approve or reject them with a mandatory rejection reason, that the host is notified in both cases, and approved listings appear immediately in customer search.			
Precondition: Admin is logged in. At least one property listing is in 'Pending Approval' status.			
Dependencies: Host-submitted listings with Pending status must exist; email notification service must be active.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Pending Listings page and observe the listing table	N/A	All pending listings are shown with listing information and host information; filter options are visible		
2. Click 'Approve' on a pending listing and confirm	Any pending listing	Listing status changes to 'Active'; host receives an email notification about the approval		

3. Verify the approved listing appears immediately in customer-facing property search	Search for approved listing as a Customer	Approved listing is visible in customer search results immediately after approval		
4. Click 'Reject' on a pending listing, enter a rejection reason, and confirm	Rejection Reason: 'Incomplete property images'	Listing is rejected; host receives an email notification that includes the rejection reason provided		
5. Attempt to reject a listing without entering a rejection reason	Rejection reason field: empty	System prevents rejection; prompts admin to provide a rejection reason before proceeding		
6. Apply the Status filter to show only Pending listings	Filter: Status = Pending	Table updates to display only listings with Pending status		
7. Apply the Category filter on the listings table	Filter: Category = Luxury	Table updates to show only Luxury category listings		
8. Apply the Date filter to narrow listings by submission date	Filter: Date = current week	Table shows only listings submitted within the selected date range		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Fariha Mahafuz Medha
Test Case ID: TC_PROP_04		Test Designed date:	17-May-2026

Test Priority: Medium		Test Executed by:	
Module Name: Property Category & Type Management Module		Test Execution date:	
Test Title: Verify that administrators can manage property categories and types			
Description: Ensure the admin can add, edit, and delete property categories and types, that changes are reflected in listing forms and search filters, and that duplicate names are rejected.			
Precondition: Admin is logged in and has navigated to the Category and Type Management page.			
Dependencies: Property listing form and customer search filter must consume categories/types from the database.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Click 'Add Category', enter a new category name, and save	Category Name: Eco-Friendly	New category is added to the category list successfully; no error is shown		
2. Attempt to add a category with a name that already exists	Category Name: Luxury (already exists)	System shows error: duplicate category name is not allowed; category is not added		
3. Edit an existing category name and save	Category: Eco-Friendly → Eco Stay	Category is updated in the list; the updated name is reflected in the host listing form and customer search filters		

4. Delete an unused category and verify it is removed	Category: Eco Stay (no listings assigned)	Category is removed from the list; it no longer appears in listing forms or search filters		
5. Add a new Property Type and verify it appears in the host listing form	Type: Cabin	New type 'Cabin' is selectable in the Property Type dropdown on the host listing form		
6. Verify updated categories are reflected in the customer-facing search filters	Open customer property search filters	Updated categories and types appear correctly in the customer-facing search filter options		
7. Attempt to add a property type with a duplicate name	Type: Hotel (already exists)	System shows error: duplicate type name is not allowed; type is not added		
8. Delete an existing property type and verify it is removed from search filters	Type: Cabin	Cabin type is removed from the list and no longer shown in customer search filter options		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Argho Dey
Test Case ID: TC_SRCH_01		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	

Module Name: Property Search & Filter Module		Test Execution date:	
Test Title: Verify that customers can search and filter properties			
Description: Ensure the search page allows filtering by location, dates, guest count, price, category, type, amenities, and ratings; updates results dynamically; supports sorting; and shows a no-result message when no properties match.			
Precondition: Customer is logged in. Multiple approved property listings with varied types, categories, and amenities exist.			
Dependencies: Approved listings with varied attributes must exist in the system.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Enter a location, check-in/check-out dates, and guest count, then click 'Search'	Location: Cox's Bazar, Check-in: next Friday, Check-out: +3 days, Guests: 2	Search results display matching property cards with image, price, rating, and location for the given criteria		
2. Apply the Price Range filter and observe results	Price Range: 2000 – 5000 BDT	Results update dynamically to show only properties within the specified price range		
3. Apply the Property Type filter — select 'Villa'	Type: Villa	Results update to show only Villa-type properties; other property types are hidden		

4. Apply the Ratings filter — select 4 stars and above	Rating: 4+	Results update to show only properties rated 4 stars or higher		
5. Sort results by 'Price: Low to High'	Sort: Price Low to High	Properties are reordered from lowest to highest price per night		
6. Search with criteria that match no available properties	Location: Remote Island X	System shows a 'No properties match your criteria' message; no blank page or error is displayed		
7. Apply the Amenities filter — select 'WiFi'	Amenity: WiFi	Results update to show only properties that include WiFi in their amenities		
8. Verify results update dynamically without a full page reload after applying a filter	Apply any filter	Search results update on the same page without requiring a full page reload		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Argho Dey
Test Case ID: TC_SRCH_02		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	

Module Name: Property Details Module		Test Execution date:	
Test Title: Verify that customers can view full property details			
Description: Ensure the property detail page shows complete information including photos, amenities, availability calendar, host info, ratings and reviews, and allows the customer to proceed to booking or add to cart.			
Precondition: Customer is logged in. At least one approved property with images, amenities, and reviews exists.			
Dependencies: Approved properties with complete data (photos, reviews, host info) must be in the system.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Click on any property card from search results	Any approved property	Property detail page loads displaying: title, images/gallery, price per night, type, category, location, and description		
2. Verify the Amenities section is displayed on the detail page	N/A	Amenities list (e.g., WiFi, AC, Parking) is visible for the selected property		
3. Check the Availability Calendar for a range of dates	Select checkin and checkout dates	Calendar displays available and unavailable dates; user can determine if selected dates are open		
4. Verify the Ratings and Reviews section is	N/A	Customer reviews with star ratings and comments are		

present on the detail page		displayed in the Reviews section		
5. Verify Host Information is shown on the property detail page	N/A	Host name and relevant host information are visible on the property detail page		
6. Click the 'Book Now' button	N/A	User is navigated to the booking/checkout flow for the selected property		
7. Click the 'Add to Cart' button	N/A	Property is added to the cart; a success notification is shown; cart item count updates in navigation		
8. Verify the property image gallery is navigable	Click through property images	Images are navigable (next/previous); the gallery loads without error		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Argho Dey
Test Case ID: TC_SRCH_03		Test Designed date:	17-May-2026
Test Priority: Medium		Test Executed by:	

Module Name: Offers & Discounts Module		Test Execution date:	
Test Title: Verify that customers can view offers and apply promo codes			
Description: Ensure the Offers page displays all available offers with relevant details, allows promo code application, validates codes before applying, updates booking price on valid codes, and shows an error for invalid or expired codes.			
Precondition: Customer is logged in. At least one valid and one expired promo code exist in the system.			
Dependencies: Active and expired promo codes must be configured in the system.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Offers & Discounts page	N/A	Page loads with a list of available offers showing: image, title, discount %, and validity date for each offer		
2. View the details of any single offer	Click on any offer card	Offer details are shown: discount amount, validity period, and applicable conditions		
3. Apply a valid promo code in the promo code input field	Promo Code: STAY20	Promo code is validated successfully; discount is applied; booking price updates to reflect the discount		

4. Apply an invalid promo code	Promo Code: WRONGCODE	System shows an error message for the invalid promo code; no discount is applied		
5. Apply an expired promo code	Promo Code: EXPIRED10	System shows an error indicating the offer has expired; no discount is applied		
6. Verify booking price updates correctly after	Promo Code: STAY20 (20%	Booking summary reflects the correctly discounted price		
applying a valid promo code	off); Total: 5000 BDT	(4000 BDT) after applying the code		
7. Verify the Featured Deals section is visible on the page	Observe page layout	Highlighted featured deal cards are present and visible on the Offers & Discounts page		
8. Verify offers are grouped into categories on the page	Observe offer categories	Offers are organized into categories: Seasonal Offers, Promo Codes, and Special Packages		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Argho Dey
Test Case ID: TC_BOOK_01		Test Designed date:	17-May-2026

Test Priority: High		Test Executed by:	
Module Name: Cart Management Module		Test Execution date:	
Test Title: Verify that customers can manage their cart			
Description: Ensure the customer can add properties to the cart, view all cart items with pricing, remove individual items, apply promo codes from the cart, and proceed to checkout.			
Precondition: Customer is logged in. At least one approved property is available for booking.			
Dependencies: Approved property listings must be available; promo code service must be functional.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Add a property to the cart from its detail page and navigate to the Cart page	Any approved property with dates selected	Cart page loads showing the added property with name, dates, guests, and price breakdown		
2. Add multiple properties to the cart and verify the cart displays all items	Add 2–3 different properties	All added properties are displayed in the cart with individual prices and a total price summary		
3. Remove one property from the cart	Remove one cart item	Property is removed from the cart; cart updates immediately; total price adjusts accordingly		

4. Apply a valid promo code from the cart page	Promo Code: STAY20	Promo code is validated; total cart price is updated to reflect the discount		
5. Apply an invalid or expired promo code from the cart	Promo Code: INVALID99	System shows an error message for the invalid/expired code; cart total remains unchanged		
6. Click 'Proceed to Checkout' from the cart page	N/A	User is navigated to the booking/payment page with the selected cart items prefilled		
7. Verify the cart is accessible from the main navigation bar	Click cart icon from any page	Cart page or dropdown is accessible from any page on the platform via the navigation bar		
8. Verify the cart item count in the navigation updates after adding a property	Add one property to an empty cart	Cart icon in the navigation bar updates to show a count of 1		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	Argho Dey
Test Case ID: TC_BOOK_02		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	

Module Name: Booking & Partial Payment Module		Test Execution date:	
Test Title: Verify that customers can create a booking with partial payment			
Description: Ensure the system allows a customer to book a property by paying a partial amount, calculates and displays the remaining balance, confirms booking with 'Partial Paid' status, and stores the booking in history.			
Precondition: Customer is logged in and has selected a property to book. Payment gateway is configured.			
Dependencies: Active property listings must be available; payment gateway integration must be functional.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Proceed to the booking page for a selected property and observe the payment section	Property: Sea View Villa, Dates: selected	Booking summary clearly shows total price, partial payment amount (e.g., 30%), and remaining balance		
2. Verify that partial payment amount and remaining balance calculations are accurate	Total Price: 10,000 BDT; Partial: 30%	Partial amount is shown as 3,000 BDT and remaining balance as 7,000 BDT		
3. Select a payment method and click 'Pay Partial Amount'	Payment Method: bKash	Partial payment is processed; booking is confirmed with status 'Partial Paid'; confirmation message is displayed		

4. Verify the booking appears in Booking History with 'Partial Paid' status	Navigate to Booking History	Booking is listed with correct property name, dates, and 'Partial Paid' status		
5. Verify the remaining balance is visible in the booking details	Open the booking detail view	Remaining balance amount is clearly shown in booking details and booking history		
6. Verify the system allows the customer to complete the remaining payment before checkin	Navigate to partially paid booking in Booking History	'Pay Remaining Amount' option is available and visible for the partially paid booking		
7. Attempt to proceed to payment without selecting a payment method	Payment method: none selected	System shows a validation error; payment cannot proceed without a payment method selected		
8. Verify booking status is visible as 'Partial Paid' in the admin panel	Check booking in admin Booking Oversight	Admin can see the booking listed with 'Partial Paid' status in the booking overview		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	MD. Naimul Haque
Test Case ID: TC_BOOK_03		Test Designed date:	17-May-2026

Test Priority: High		Test Executed by:	
Module Name: Remaining Payment Module		Test Execution date:	
Test Title: Verify that customers can complete the remaining payment for a booking			
Description: Ensure the system displays remaining balance for partial-paid bookings, processes the remaining payment, updates status to 'Fully Paid', sends a confirmation notification, and prevents overpayment or duplicate payment.			
Precondition: Customer is logged in with at least one booking in 'Partial Paid' status.			
Dependencies: A partially paid booking must exist; payment gateway must be functional.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to Booking History and open a 'Partial Paid' booking	Booking Status: Partial Paid	Booking detail page shows: paid amount, remaining balance, and 'Pay Remaining Amount' button		
2. Click 'Pay Remaining Amount', select a payment method, and confirm	Payment Method: Card; Remaining Balance: 7,000 BDT	Remaining payment is processed; booking status updates to 'Fully Paid'; payment confirmation notification is generated		
3. Verify booking history is updated to 'Fully Paid' status after payment	Check Booking History after full payment	Booking record shows 'Fully Paid' status with an updated payment summary		

4. Verify a payment confirmation notification is received after completing full payment	Complete remaining payment	Email or in-app notification is sent confirming full payment and updated booking status		
5. Attempt to pay an amount greater than the remaining balance	Enter amount: 9,000 BDT (remaining balance is 7,000 BDT)	System blocks the transaction; shows error: amount exceeds remaining balance		
6. Attempt duplicate payment on a 'Fully Paid' booking	Try to pay again on an already fully paid booking	'Pay Remaining Amount' button is disabled or hidden for fully paid bookings; duplicate payment is prevented		
7. Verify the payment method selection is available on the remaining payment page	Observe the remaining payment page	At least one payment method option is displayed and selectable on the payment page		
8. Verify the booking detail page clearly shows payment status	Open any booking in Booking History	Payment status (Partial Paid / Fully Paid) is clearly labelled in the booking detail view		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	MD. Naimul Haque
Test Case ID: TC_BOOK_04		Test Designed date:	17-May-2026

Test Priority: Medium		Test Executed by:	
Module Name: Booking History Module		Test Execution date:	
Test Title: Verify that customers can view their booking history			
Description: Ensure the Booking History page lists all bookings categorized as past and upcoming, displays property name, dates, price, payment status, booking status, and supports filtering/sorting and detail view.			
Precondition: Customer is logged in with at least one past and one upcoming booking.			
Dependencies: Completed and upcoming bookings must exist for the test customer account.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Booking History page	N/A	Page loads showing a list of all customer bookings with: property name, dates, price, payment status, and booking status		
2. Verify bookings are categorized into 'Past Bookings' and 'Upcoming Bookings'	Observe page sections	Separate sections or tabs for past and upcoming bookings are clearly visible on the page		
3. Verify payment status is displayed for each booking entry	Observe the booking list	Each booking entry shows payment status: Partial Paid or Fully Paid		

4. Click 'View Details' for any booking	Any existing booking	Booking detail view opens with complete information: property, dates, price, booking status, and payment summary		
5. Apply filter by Status — select 'Upcoming'	Filter: Status = Upcoming	Booking list updates to show only upcoming bookings		
6. Sort bookings by date — most recent first	Sort: Date Descending	Booking list reorders to display the most recent bookings at the top		
7. Verify the Booking History page is accessible from the customer dashboard navigation	Navigate from Customer Dashboard	Booking History is reachable directly from the customer dashboard or navigation menu		
8. Verify all required columns are visible in the booking history table	Observe the booking table	Table shows columns: Property Name, Dates, Price, Payment Status, Booking Status, and Actions		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	MD. Naimul Haque
Test Case ID: TC_REV_01		Test Designed date:	17-May-2026
Test Priority: Medium		Test Executed by:	

Module Name: Customer Review Submission Module		Test Execution date:	
Test Title: Verify that customers can submit a property review			
Description: Ensure a customer who completed a stay can submit a star rating and required review text, optionally upload images, and receive a confirmation message upon successful submission.			
Precondition: Customer is logged in with at least one completed (past) booking for a property.			
Dependencies: A completed booking must exist; review submission module must be active.			

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Submit Review page for a completed booking	Property: Sea View Villa (completed stay)	Review form loads with: property name auto-filled, 1–5 star rating selector, comment/review text box, and optional image upload field		
2. Submit the review without entering any comment text	Rating: 3 stars, Comment: empty	System shows validation error: review text is required; submission is blocked		
3. Submit the review without selecting any star rating	Rating: none, Comment: 'Good place'	System shows validation error: star rating is required; submission is blocked		
4. Select a star rating, enter review text, and click 'Submit Review'	Rating: 4 stars, Comment: 'Great stay, very clean!'	Review is submitted successfully; confirmation message is displayed after submission		

5. Submit review with an optional image attached	Image: room_photo.jpg (2MB), Rating: 5, Comment: 'Excellent!'	Review with image is submitted; confirmation message shown; review appears on property page		
6. Submit review without uploading any image	No image, Rating: 4, Comment: 'Nice property'	Review is submitted successfully without an image; image upload field is confirmed as optional		
7. Verify the submitted review appears on the property detail page	Navigate to the reviewed property as any user	Review with rating, comment, and customer name is displayed in the Reviews section		
8. Verify the star rating selector allows values from 1 to 5	Observe the rating control on the review form	Rating selector allows selection of 1, 2, 3, 4, or 5 stars		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	MD. Naimul Haque
Test Case ID: TC_REV_02		Test Designed date:	17-May-2026
Test Priority: Medium		Test Executed by:	
Module Name: Host Review Response Module		Test Execution date:	

Test Title: Verify that hosts can reply to customer reviews
Description: Ensure the host can view all customer reviews, submit a reply, edit or delete that reply, and have all changes reflected instantly under the original review.
Precondition: Host is logged in. At least one customer review exists for the host's property.
Dependencies: At least one approved customer review must be present for a host-owned property.

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Review Responses page and observe the review list	N/A	All customer reviews are displayed with: Customer Name, Rating, Review Text, Date, and a Reply Box under each review		
2. Type a reply in the reply box for a review and click 'Submit Reply'	Reply: 'Thank you for your feedback!'	Reply is submitted and displayed immediately under the original customer review		
3. Verify the reply is displayed instantly below the original review without page reload	Observe after submitting the reply	Reply appears under the customer's review immediately; no full page reload is required		
4. Click 'Edit Reply' on a previously submitted reply and update the content	Updated Reply: 'We appreciate your kind words!'	Reply content is updated; changes are reflected immediately on the review list		
5. Click 'Delete Reply' on an existing reply and confirm deletion	Any existing host reply	Reply is removed; the review section updates without the deleted reply		

6. Verify a customer can see the host's reply under their review on the property page	View property detail page as a customer	Host reply is displayed beneath the customer's review on the public property detail page		
7. Submit a blank reply (no text entered) for a review	Reply text: empty	System shows a validation error; blank reply is not submitted		
8. Verify the host can view reviews from all owned properties on the same page	Host with 2+ properties	Reviews from all host-owned properties are visible together on the Review Responses page		

Test case table of functional requirements

Project Name: Stay Finder		Test Designed by:	MD. Naimul Haque
Test Case ID: TC_REV_03		Test Designed date:	17-May-2026
Test Priority: High		Test Executed by:	
Module Name: Admin Review Moderation Module		Test Execution date:	
Test Title: Verify that admin can perform review moderation			
Description: Ensure the admin can view all submitted reviews, approve or reject them with immediate status update, delete inappropriate reviews, and that only approved reviews are visible to users.			

Precondition: Admin is logged in. Reviews with Pending, Approved, and Rejected statuses exist in the system.

Dependencies: Customer-submitted reviews in various statuses must exist in the system.

Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Navigate to the Review Moderation	N/A	Table displays all reviews with: Review ID, User Name, Property Name, Rating,		
page and observe the review table		Comment, and Status (Pending/Approved/Rejected); filter options are visible		
2. Click 'Approve' on a Pending review	Any Pending review	Review status changes to 'Approved'; the review becomes visible to customers on the property page		
3. Click 'Reject' on a Pending review	Any Pending review	Review status changes to 'Rejected'; the review is not visible to customers		
4. Verify only Approved reviews are visible on the property detail page for customers	View property page as a Customer	Only reviews with 'Approved' status are shown; Pending and Rejected reviews are hidden from public view		
5. Click 'Delete Review' on an inappropriate review	Any review (any status)	Review is permanently deleted and no longer appears in the admin review moderation table		

6. Apply the Status filter — select 'Pending'	Filter: Status = Pending	Table updates to show only reviews with Pending status		
7. Verify the review status label updates immediately after an approval or rejection action	Approve a Pending review	Review status label changes from 'Pending' to 'Approved' instantly without requiring a page reload		
8. Apply the Date filter to narrow down reviews by submission date	Filter: Date = last 7 days	Table shows only reviews submitted within the selected date range		

7. CONCLUSION

In this project, StayFinder is designed as a comprehensive travel stay platform designed to bridge the gap between travelers and property owners. StayFinder's benefits include providing customers with affordable, reliable accommodations through secure payments, while offering hosts robust tools to manage listings and track earnings.

Throughout the documentation, we established clear project scopes covering profile, user, booking, and property management. We developed detailed user stories and a comprehensive Requirement Traceability Matrix to ensure all functional requirements for customers, hosts, and super admins are met. The workflow integrated industry-standard practices, including Git for version control, Trello for task management, and Figma for the intuitive UI/UX design.

In the future, we aim to expand the platform by integrating AI-driven personalized recommendations and advanced analytics for hosts. By maintaining these high standards of software engineering, StayFinder is well-positioned to become a reliable, centralized solution for the modern travel industry.